



DARKNESS'S EMBRACE



UNIVERSITÀ
DEGLI STUDI DI BARI
ALDO MORO

Laboratorio di Informatica A.A. 2022/2023

Caso Studio realizzato da G. Giovanni Piccinno e Francesco Scialpi

Indice

1. Analisi	3
1.1 Specifiche del problema – Traccia Caso Studio 4	3
1.2 Analisi specifiche del problema	4
1.2.1 Avventura Testuale	4
1.2.2 Gioco di Ruolo	4
1.2.3 Ambientazione Immersiva e Misteriosa	4
1.2.4 Struttura di Gioco	4
1.2.5 Sezione Admin e Cheat	5
1.3 Requisiti funzionali	6
1.4 Casi d'uso	10
2. Progettazione	39
2.1 Costanti	39
2.2 Strutture Dati	42
2.3 File	44
2.3.1 File di Salvataggio	44
2.3.2 File Audio	44
2.3.2 File Dati	45
2.3.3 File ASCII art	46
2.4 Librerie e Funzioni	47
2.5 Dipendenze tra funzioni	51
2.6 Flow Charts	54
3. Codifica	63
3.1 Opzione “Avanza” del menu di gioco e le funzioni: advance_map1_arrows() / advance_map2_arrows()	63
3.2 Funzione: print_wrapped_text()	65
3.3 Interfaccia Grafica e Audio	66
3.3.1 Funzione render_txt()	66
3.3.2 Funzione render_map1()	67
3.3.2 Funzione PlaySound()	67
3.4 Funzione: insert_password()	68
3.5 Funzione: has_uppercase_number()	69
4. Test	70
4.1 Come è stata condotta la fase di Testing	70
5. Conclusioni	75
5.1 Conclusioni	75
5.2 Possibili Future Implementazioni	75
5.3 Bug Noti	75
6. Guida Utente	76
6.1 Soluzioni agli eventi di gioco	76
6.2 Altre Informazioni	76

1. Analisi

1.1 Specifiche del problema – Traccia Caso Studio 4

Un team di sviluppatori desidera creare un gioco di avventura testuale in stile retrò, ispirato ai classici giochi di avventura testuale degli anni Ottanta e Novanta.

Il gioco dovrà presentare una trama avvincente e coinvolgente, con una serie di scene o luoghi in cui il giocatore può muoversi, interagire con i personaggi e risolvere enigmi per progredire nella storia.

Il sistema deve prevedere la creazione e gestione del contenuto del gioco, compresa la progettazione della trama, dei personaggi, delle ambientazioni e degli enigmi. Dovrà inoltre consentire la creazione di dialoghi tra i personaggi, la gestione degli oggetti nel gioco e la gestione degli eventi che si verificano durante il gioco.

Il gioco dovrà includere un'interfaccia utente intuitiva che consenta ai giocatori di interagire con il gioco tramite comandi di testo, come la digitazione di azioni o scelte da compiere. Dovrà inoltre essere possibile salvare e caricare lo stato del gioco in qualsiasi momento.

Il sistema dovrà essere in grado di gestire la logica di gioco, come controllare il flusso della trama in base alle scelte del giocatore, gestire gli oggetti nel gioco e valutare le condizioni per risolvere gli enigmi o avanzare nella storia.

Infine, il sistema dovrà includere un sistema di gestione dei punteggi e dei progressi del giocatore, che consenta di tenere traccia dei punti guadagnati, degli obiettivi raggiunti e delle sfide completate.

1.2 Analisi specifiche del problema

Il nostro gruppo si è posto nei panni di un piccolo studio di sviluppo. L'approccio allo sviluppo del videogioco è stato progressivo analizzando inizialmente tutti gli aspetti necessari a soddisfare l'ipotetico produttore del videogioco che ha dato le linee guida su come debba essere il prodotto, cioè la traccia del caso studio ed in seguito analizzando tutti gli aspetti che, come sviluppatori e game designers, abbiamo voluto aggiungere per rendere il videogioco unico.

1.2.1 Avventura Testuale

Il programma deve riprodurre un videogioco di genere avventura testuale. La trama e le interazioni all'interno del gioco sono principalmente basate su testo scritto, con descrizioni dettagliate dell'ambiente, degli oggetti e delle situazioni. La sfida principale per il giocatore consiste nel risolvere enigmi, battaglie e prendere decisioni attraverso l'uso di comandi testuali. Il giocatore interagisce con il gioco attraverso una riga di comando, in cui inserisce i comandi desiderati utilizzando una sintassi specifica.

1.2.2 Gioco di Ruolo

Volendo creare un videogioco immersivo e coinvolgente che andasse oltre una successione di enigmi, abbiamo voluto dare un'impronta ruolistica al videogioco che quindi gestirà una serie di statistiche legate allo sviluppo del personaggio, un battle system, un sistema di equipaggiamento, l'uso di oggetti consumabili ed incontri con dialoghi personalizzati in base ad eventi pregressi.

1.2.3 Ambientazione Immersiva e Misteriosa

Per Darkness's Embrace è stato scelto un mondo fantasy avvolto da un'atmosfera dark e misteriosa, ispirato dai giochi di Hdetaka Miyazaki che a sua volta ha spesso preso ispirazione dall'opera di Kentaro Miura, Berserk. Abbiamo voluto eliminare i cliché convenzionali. I personaggi che si incontreranno avranno relazioni complesse per offrire un'esperienza narrativa profonda. Si potranno scoprire segreti nascosti attraverso dialoghi e oggetti, mentre ogni dettaglio dell'ambientazione svelerà informazioni cruciali per comprendere appieno il mondo di gioco.

1.2.4 Struttura di Gioco

Il gioco si svilupperà in due atti principali, corrispondenti all'esplorazione delle due ambientazioni principali del gioco. Ci saranno anche degli atti secondari che fungeranno da introduzione al mondo di gioco comprendendo la creazione del personaggio, un intermezzo che condurrà il giocatore da un'ambientazione all'altra ed un epilogo. Il giocatore potrà inoltre interagire con il mondo di gioco tramite l'esplorazione degli ambienti, la risoluzione di enigmi ambientali, dialoghi con npc e lottando contro nemici.

1.2.5 Sezione Opzioni

L'obiettivo principale nell'introdurre questo menu è di dare, agli utenti che ne richiedono l'accesso, il controllo dell'esperienza di gioco. Nel menu opzioni ci si potrà registrare per accedere a tale area che permetterà una vasta gamma di opzioni che vi permetteranno di adattare il gioco alle preferenze migliori. Potrete per lo più modificare le statistiche del giocatore, i suoi consumabili e decidere se sbloccare gli equipaggiamenti oppure no.

1.3 Requisiti funzionali

**NOTA:* Il codice è scritto in forma RXYX dove X indica dove è presente la funzionalità e YY il l'ordine delle stesse.

M: main.cpp U:utility.cpp G:game.cpp I:interactions.cpp B:battle.cpp A:admin.cpp

Codice*	Nome	Descrizione
RM00	Preparazione all'Avvio	Allocazione della memoria necessaria per l'uso dei dati giocatore e dei nemici.
RM01	Nuovo Gioco	Avvia una nuova partita.
RM02	Continua	Fa selezionare un salvataggio e fa riprendere il gioco da dove si era interrotto.
RM03	Sezione Admin	Permette ad un utente provvisto di credenziali di modificare i file di salvataggio.

RU01	Stampa Testo Carattere per Carattere	Prende una stringa e la ne stampa carattere per carattere con un ritardo impostato in fase di sviluppo.
RU02	Stampa Texture ASCII	Legge un file sul quale vi è un'immagine in ASCII art e la stampa.
RU03	Stampa su Righe	Prende una stringa e la stampa su righe di cento caratteri massimo.

RB01	Caricamento Nemico	Legge da file i dati del nemico che il giocatore sta per affrontare e li carica in memoria.
RB02	Inizializzazione Valori Battaglia	Prepara i valori temporanei e non utili per affrontare la battaglia.
RB03	Interfaccia Menù Battaglia	Stampa a schermo il menù durante una battaglia.
RB04	Tabella delle Statistiche	Legge i dati del giocatore e li stampa a schermo in una tabella elaborata in ASCII art.
RB05	Interfaccia Consumabili	Legge dai dati del giocatore le quantità degli oggetti consumabili disponibili e li mostra.
RB06	Menù Consumabili in Battaglia	Elabora la scelta del giocatore di utilizzare un oggetto consumabile, aggiornando le statistiche e le quantità temporanee relative alla battaglia.
RB07	Aggiornamento Consumabili	Aggiorna i valori non temporanei dei consumabili con quelli temporanei utilizzati in battaglia.
RB08	Sistema di Loot/Punteggio post Battaglia	Elabora gli oggetti che vengono dati al giocatore al termine della battaglia ed i relativi punti.

RB09	Calcolo Danno PvE	Calcola il danno che il giocatore infligge al nemico tenendo conto della penetrazione rispetto alla difesa nemica.
RB10	Calcolo Danno EvP	Calcola il danno che il nemico infligge al giocatore tenendo conto di un danno minimo rispetto ad una difesa teorica del 100%
RB11	Calcolo Danni in Difesa	Calcola il danno che il giocatore riceve quando si pone in modo difensivo.
RB12	Turno di Gioco	Stabilisce in base alle statistiche chi inizia ad attaccare e calcola l'eventuale colpo critico.
RB13	Battaglia	Legge i dati in memoria del nemico, prepara i dati del giocatore per la battaglia, gestisce il menù battaglia e le sotto funzionalità della battaglia.
RB14	Caricamento Battaglia	Legge la posizione del giocatore ed in base ad essa legge il file del nemico che si trova in tale posizione e lo carica in memoria e ne avvia la battaglia.

RG01	Menu di Avvio	Elabora la schermata principale e il relativo menù.
RG02	Descrizione della Stanza	Legge la posizione del giocatore ed in base ad essa stampa a schermo la relativa descrizione.
RG03	Interfaccia Equipaggiamento	Elabora la scelta del giocatore di utilizzare un oggetto consumabile, aggiornando le statistiche e le quantità temporanee relative alla battaglia.
RG04	Aggiunta Equipaggiamento	Aggiorna i valori non temporanei dei consumabili con quelli temporanei utilizzati in battaglia.
RG05	Rimozione Equipaggiamento	Legge da file i valori dell'equipaggiamento ed aggiorna le statistiche del giocatore.
RG06	Menù Equipaggiamento	Elabora l'interfaccia, l'aggiunta e la rimozione dell'equipaggiamento e controlla che il giocatore non indossi più pezzi della stessa tipologia di equipaggiamento.
RG07	Menù Consumabili in Esplorazione	Utilizza la stessa elaborazione dei consumabili in battaglia senza utilizzare i dati temporanei.
RG08	Gestione delle Interazioni	Legge la posizione del giocatore ed avvia l'eventuale evento legato alla sua posizione.
RG09	Inizializzazione dati Giocatore	Pone a 0 tutti i valori dei dati allocati in memoria relativi al giocatore.
RG10	Scelta Nome Giocatore	Scrive in memoria il nome scelto dal giocatore per il proprio personaggio.
RG11	Selezione Classe Giocatore	Permette la scelta della Classe iniziale di gioco e scrive nei dati relativi alla classe scelta nella memoria allocata per il giocatore.
RG12	Imposta Posizione Iniziale	Scrive nella memoria allocata per il giocatore i dati sulla posizione iniziale.
RG13	Atto 0	Elabora l'introduzione del gioco, scelta della classe, nome giocatore e prepara i dati per il primo atto.

RG14	Stampa Mappa Catacombe	Legge la posizione del giocatore e in base ad essa re-indirizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
RG15	Controllo Mura Mappa Catacombe	Legge la posizione che il giocatore vuole intraprendere e controlla se ci si può muovere verso di essa, ritornando un valore booleano.
RG16	Avanzamento Testuale Mappa Catacombe	Elabora il movimento del giocatore nella mappa tramite testo.
RG17	Avanzamento con Freccie Mappa Catacombe	Elabora il movimento del giocatore nella mappa tramite l'uso delle frecce.
RG18	Controllo Stanza Mappa Catacombe	Legge la posizione del giocatore e controlla qual è la stanza in cui si trova.
RG19	Atto 1	Elabora l'intero atto: il menù di gioco durante l'esplorazione della mappa con le relative funzionalità e la fine dell'atto dopo la sconfitta del boss locale.
RG20	Imposta Posizione Atto 2	Scrive nella memoria allocata per il giocatore i dati sulla posizione iniziale per l'atto 2.
RG21	Evento Intermezzo	Elabora l'incontro con un NPC che permetterà al giocatore l'avanzamento all'atto 2.
RG22	Atto Intermezzo	Elabora l'intermezzo del gioco, musica, testi, immagini dal post atto 1, l'evento intermezzo fino alla preparazione dei dati per l'atto 2.
RG23	Stampa Mappa Castello	Legge la posizione del giocatore e in base ad essa renderizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
RG24	Controllo Mura Mappa Castello	Legge la posizione che il giocatore vuole intraprendere e controlla se ci si può muovere verso di essa, ritornando un valore booleano.
RG25	Avanzamento Testuale Mappa Castello	Elabora il movimento del giocatore nella mappa tramite testo.
RG26	Avanzamento con Freccie Mappa Castello	Elabora il movimento del giocatore nella mappa tramite l'uso delle frecce.
RG27	Controllo Stanza Mappa Castello	Legge la posizione del giocatore e controlla qual è la stanza in cui si trova.
RG28	Atto 2	Elabora l'intero atto: il menù di gioco durante l'esplorazione della mappa con le relative funzionalità e la fine dell'atto dopo la sconfitta del boss locale.
RG29	Atto Epilogo	Elaborazione del testo, immagini, musica e crediti dell'epilogo di gioco.

RI01-10	Casse	Elabora l'interazione del giocatore con una delle dieci casse presenti nel gioco.
RI11-20	Eventi	Elabora l'interazione del giocatore con uno dei dieci eventi presenti nel gioco.

RA01	Menù Opzioni	Elabora la schermata admin e il suo relativo menù.
RA02	Inserimento password	Legge la password inserita dall'utente.
RA03	Controllo numeri e maiuscole	Controlla se la password inserita dall'utente contiene i vari parametri richiesti.
RA04	Autenticazione utente	Legge dal file se l'username e la password inseriti dall'utente coincidono con il file.
RA05	Salvataggio dei dati	Carica i dati dell'utente all'interno del file.
RA06	Caricamento dei dati	Preleva dal file i dati dell'utente.
RA07	Salvataggio nascosto dei dati	Carica direttamente i dati dell'utente all'interno del file, senza i passaggi dai vari menù e richieste.
RA08	Menu modifiche dati	Elabora la schermata per la scelta dei dati da modificare.
RA09	Modifiche delle statistiche	Modifica le statistiche del personaggio in base alla scelta dell'amministratore.
RA10	Modifiche dei consumabili	Modifica i consumabili del personaggio in base alla scelta dell'amministratore.
RA11	Modifiche degli equipaggiamenti	Modifica gli equipaggiamenti del personaggio. L'amministratore può decidere se rendere disponibili la/le arma/i oppure no.
RA12	Tabella per le statistiche	Tabella per la rappresentazione delle statistiche.
RA13	Tabella per i consumabili	Tabella per la rappresentazione dei consumabili.

1.4 Casi d'uso

Prima di leggere i vari casi d'uso è importante sapere che:

- 1) Ove non necessario, le sezioni "scenario alternativo" e "fallimento" possono essere state omesse.
- 2) Tutte le funzionalità che modificano o leggono i dati riguardanti i nemici e del giocatore richiedono l'allocazione ed inizializzazione in memoria di una struttura di tipo character o enemy. Queste strutture racchiudono tutte le possibili informazioni aggiornate in tempo reale del giocatore o del nemico. L'allocazione di tali strutture è vincolata all'avvio del gioco e come da requisito funzionale codice RMoo un eventuale fallimento causerebbe il crash del gioco; quindi, è implicito che l'allocazione in memoria delle strutture sia una preconditione condivisa da tutte queste funzionalità.
- 3) Il programma è altamente modulare di conseguenza molte funzioni hanno il compito gestire più funzioni, il requisito funzionale codice RGo6, per esempio, per fare tutto ciò che deve si affida ai requisiti funzionali codice RGo3/RGo4/RGo5, in tal caso viene omesso che si possono avere molteplici scenari alternativi in base ai problemi che le funzioni dipendenti possano dare. Ciò viene omesso per ogni requisito funzionale che dipende da ulteriori requisiti. Per maggiori informazioni sulle dipendenze si può vedere il relativo schema.

Codice	Nome	Descrizione
RMoo	Preparazione all'Avvio	Allocazione della memoria necessaria per l'uso dei dati giocatore e dei nemici.
Pre-condizioni	Avvio del programma.	
Post-condizioni	Successo: L'allocazione avviene con successo. Fallimento: L'allocazione non avviene.	
Scenario di base	Il programma si avvia e viene mostrato la schermata del titolo.	
Scenario alternativo	Il gioco va in crash all'avvio poiché non riesce ad allocare la memoria.	

Codice	Nome	Descrizione
RMo1	Nuovo Gioco	Avvia una nuova partita.
Pre-condizioni	Allocazione della memoria avvenuta con successo.	
Post-condizioni	Successo: Avvio dell'Atto Introduttivo del gioco. Fallimento: Non viene avviata la partita.	
Scenario di base	Una volta scelto il nuovo gioco, la partita ha inizio.	
Scenario alternativo	Potrebbe non avviarsi la partita rimanendo nel menù, dando seguito ad un loop o ad un crash.	

Codice	Nome	Descrizione
RM02	Continua	Fa selezionare un salvataggio e fa riprendere il gioco da dove si era interrotto.
Pre-condizioni	Caricamento dei dati utente all'interno del gioco.	
Post-condizioni	Successo: la partita continua dal punto in cui si ha salvato; Fallimento: Non viene avviata la partita.	
Scenario di base	Una volta scelto lo slot di salvataggio, il gioco riprende dal punto in cui si ha salvato.	
Scenario alternativo	Potrebbe non avviarsi la partita rimanendo nel menù, dando seguito ad un loop o ad un crash.	

Codice	Nome	Descrizione
RM03	Sezione Admin	Permette ad un utente provvisto di credenziali di modificare i file di salvataggio.
Pre-condizioni	Apertura del menù con le varie scelte.	
Post-condizioni	Successo: l'utente si iscrive/accede ed ha gli accessi per le modifiche ai dati del giocatore; Fallimento: Potrebbero esserci problemi con gli input inseriti dall'utente, dando seguito ad un crash.	
Scenario di base	Si presenta un menu con tre scelte, ovvero: Iscriviti, Accedi, Esci.	

Codice	Nome	Descrizione
RU01	Stampa Testo Carattere per Carattere	Prende una stringa e la stampa carattere per carattere con un ritardo impostato in fase di sviluppo.
Pre-condizioni	Creazione di una stringa e impostazione del delay tra la stampa di un carattere e l'altro.	
Post-condizioni	Successo: La stringa viene stampata secondo le impostazioni inserite.	
Scenario di base	A schermo si vede la stringa stampata carattere per carattere.	
Scenario alternativo	Non è previsto uno scenario alternativo perché questa funzionalità è a cura dello sviluppatore che la integra dopo aver già definito in precedenza una stringa e all'interno di funzioni, nel caso di stringa mancante il compilatore non permette la compilazione del programma.	

Codice	Nome	Descrizione
RU02	Stampa Texture ASCII	Legge un file sul quale vi è un'immagine in ASCII art e la stampa.
Pre-condizioni	L'esistenza di un file .txt nel quale è presente un'immagine ASCII art e il suo nome posto all'interno di una stringa che verrà utilizzato dalla funzione.	
Post-condizioni	Successo: L'immagine presente nel file viene stampata correttamente a schermo. Fallimento: L'immagine non viene stampata.	
Scenario di base	L'immagine viene stampata a schermo e il programma prosegue normalmente.	
Scenario alternativo	Nel caso in cui il file non venisse letto per vari motivi, viene visualizzato a schermo un errore e il programma continua ad essere eseguito.	

Codice	Nome	Descrizione
RU03	Stampa su righe	Prende una stringa e la stampa in righe da cento caratteri massimo.
Pre-condizioni	L'esistenza di una stringa che viene passata alla funzione.	
Post-condizioni	Successo: La stringa viene stampata correttamente in righe.	
Scenario di base	La stringa viene stampata correttamente in righe, senza che le parole vengano tagliate.	

Codice	Nome	Descrizione
RBo1	Caricamento Nemico	Legge da file i dati del nemico che il giocatore sta per affrontare e li carica in memoria.
Pre-condizioni	L'esistenza del file .txt sul quale vi siano i dati del nemico e l'avvenuta allocazione della memoria per il nemico.	
Post-condizioni	Successo: La battaglia si avvia normalmente Fallimento: La battaglia si avvia dopo un messaggio di errore.	
Scenario di base	La battaglia si avvia secondo i parametri previsti dal file.	
Scenario alternativo	In caso di problemi con l'apertura o lettura del file, il programma stampa un errore a riguardo, essendo comunque allocata la memoria per il nemico e inizializzata, la battaglia si avvierà con tutti i parametri impostati a zero.	

Codice	Nome	Descrizione
RBo2	Inizializzazione Valori Battaglia	Prepara i valori temporanei e no, utili per affrontare la battaglia.
Pre-condizioni	Allocazione della memoria per i dati del giocatore.	
Post-condizioni	Successo: I valori definitivi delle statistiche vengono copiati nelle statistiche temporanee.	
Scenario di base	L'utente non si accorge dell'avvio di tale funzionalità e il programma prosegue.	
Scenario alternativo	Nel caso di problemi in fase di copia, le statistiche potrebbero creare degli scenari inaspettati su tutte le funzioni che le utilizzano.	

Codice	Nome	Descrizione
RBo3	Interfaccia Menù Battaglia	Stampa a schermo il menù durante una battaglia.
Pre-condizioni	Avvio di una Lotta	
Post-condizioni	Successo: Il menù viene stampato correttamente a schermo.	
Scenario di base	Il menù viene stampato correttamente a schermo ed il programma richiede l'intervento dell'utente per la scelta da effettuare.	

Codice	Nome	Descrizione
RBo4	Tabella delle Statistiche	Legge i dati del giocatore e li stampa a schermo in una tabella elaborata in ASCII art.
Pre-condizioni	Allocazione della memoria per i dati del giocatore.	
Post-condizioni	Successo: La tabella delle statistiche viene stampata a schermo.	
Scenario di base	La tabella delle statistiche viene stampata a schermo.	
Scenario alternativo	Nel caso di problemi con la copia/modifiche di statistiche potrebbero esserci valori inaspettati stampati a schermo.	

Codice	Nome	Descrizione
RBo5	Interfaccia Consumabili	Legge dai dati del giocatore le quantità degli oggetti consumabili disponibili e li mostra
Pre-condizioni	Allocazione della memoria per i dati del giocatore.	
Post-condizioni	Successo: L'interfaccia viene stampata a schermo.	
Scenario di base	L'interfaccia viene stampata a schermo con le relative quantità degli oggetti corrette.	
Scenario alternativo	Nel caso di problemi con la copia/modifica dei consumabili potrebbero esserci valori inaspettati stampati a schermo.	

Codice	Nome	Descrizione
RBo6	Menù Consumabili in Battaglia	Elabora la scelta del giocatore di utilizzare un oggetto consumabile, aggiornando le statistiche e le quantità temporanee relative alla battaglia.
Pre-condizioni	Allocazione della memoria per i dati del giocatore.	
Post-condizioni	Successo: Dopo aver scelto quale consumabile utilizzare, il programma ne riduce la quantità di uno e aggiorna le statistiche in base al consumabile. Fallimento: Nel caso in cui non ci dovesse essere disponibilità del consumabile, il programma avvisa il non riconoscimento della scelta e richiede un nuovo input.	
Scenario di base	Il giocatore sceglie un consumabile, ne scrive il nome e il gioco applica i relativi potenziamenti.	

Codice	Nome	Descrizione
RBo7	Aggiornamento Consumabili	Aggiorna i valori non temporanei dei consumabili con quelli temporanei utilizzati in battaglia.
Pre-condizioni	Allocazione della memoria per i dati del giocatore.	
Post-condizioni	Successo: La copia dei valori temporanei nei non temporanei avviene con successo. Fallimento: La copia dei valori temporanei nei non temporanei non avviene correttamente.	
Scenario di base	La copia dei valori temporanei nei non temporanei avviene senza che il giocatore si accorga di nulla.	
Scenario alternativo	Un eventuale problema nella copia si noterebbe in seguito nel conteggio dei consumabili ottenuti.	

Codice	Nome	Descrizione
RBo8	Sistema di Loot/Punteggio post Battaglia	Elabora gli oggetti che vengono dati al giocatore al termine della battaglia ed i relativi punti.
Pre-condizioni	Vittoria di una Lotta	
Post-condizioni	Successo: Vengono simulati 6 lanci di un dado a 6 facce, un lancio per ogni consumabili esistente, nel caso in cui il dado faccia 1 l'oggetto viene concesso al giocatore. Al termine dei lanci viene calcolato il punteggio relativo alla battaglia a partire da una base di 50 punti ed un punto di penalità per ogni turno utilizzato per sconfiggere il nemico.	
Scenario di base	Il giocatore vince la Lotta e vede a schermo gli oggetti e il punteggio ottenuti.	
Scenario alternativo	Nel caso di una lotta superiore di durata superiore a 50 turni, potrebbe perdere punti e anche non ottenere alcun consumabile.	

Codice	Nome	Descrizione
RBo9	Calcolo Danni PvE	Calcola il danno che il giocatore infligge al nemico tenendo conto della penetrazione rispetto alla difesa nemica.
Pre-condizioni	Inizio del Turno di Battaglia	
Post-condizioni	Successo: Il danno viene calcolato utilizzando i valori di difesa come percentuali di penetrazione, viene calcolato sia il danno fisico che quello arcano inflitto e sommato.	
Scenario di base	Il giocatore una volta terminato il turno vedrà a schermo il risultato dell'attacco.	

Codice	Nome	Descrizione
RB10	Calcolo Danni EvP	Calcola il danno che il nemico infligge al giocatore tenendo conto di un danno minimo rispetto ad una difesa teorica del 100%
Pre-condizioni	Inizio del Turno di Battaglia	
Post-condizioni	Successo: Il danno viene calcolato utilizzando i valori di difesa come percentuali di penetrazione, viene calcolato sia il danno fisico che quello arcano inflitto e sommato. In questo caso però viene applicato un danno base pari a 20 poiché il giocatore potrebbe ottenere nel gioco una difesa pari a 100 corrispondente all'azzeramento della penetrazione del danno.	
Scenario di base	Il giocatore una volta terminato il turno vedrà a schermo il risultato dell'attacco.	

Codice	Nome	Descrizione
RB11	Calcolo Danni in Difesa	Calcola il danno che il giocatore riceve quando si pone in modo difensivo.
Pre-condizioni	Scelta di mettersi in posizione difensiva durante una battaglia.	
Post-condizioni	Successo: Viene calcolato il danno EvP e successivamente dimezzato.	
Scenario di base	Il giocatore vede a schermo il danno ricevuto in difesa.	

Codice	Nome	Descrizione
RB12	Turno di Gioco	Stabilisce in base alle statistiche chi inizia ad attaccare e calcola l'eventuale colpo critico.
Pre-condizioni	Scelta di attaccare un nemico in battaglia.	
Post-condizioni	Successo: Viene effettuato un confronto tra la velocità del giocatore e del nemico per stabilire chi inizierà a lottare per primo, viene lanciato un dado a 10 facce per simulare una probabilità del 10% di ottenere e/o ricevere un colpo critico, viene calcolato il danno tramite i relativi calcolatori ed in caso di critico maggiorato del 20%. Viene aumentato il contatore dei turni. In caso i danni infliggano maggior punti vita rispetto a quelli disponibili essi vengono riportati a 0.	
Scenario di base	Il turno avviene con successo e a schermo il giocatore visualizza il resoconto del turno.	

Codice	Nome	Descrizione
RB13	Battaglia	Legge i dati in memoria del nemico, prepara i dati del giocatore per la battaglia, gestisce il menù battaglia e le sotto funzionalità della battaglia.
Pre-condizioni	Avvio di una scena di Lotta contro un nemico presente in una stanza e corretta copia dei dati da file nella memoria del nemico.	
Post-condizioni	Successo: Viene renderizzata a schermo l'immagine del nemico, viene posto un loop che fa continuare la lotta fino a quando gli hp del nemico non vengono azzerati con un menù che dà la libertà di agire al giocatore come meglio crede ogni turno, tra cui anche l'opzione di fuga. Nel caso in cui gli hp del giocatore vadano a zero, l'accesso di ogni menù viene precluso e gli viene concesso di poter effettuare il caricamento di un salvataggio. Fallimento: In caso di problemi nella fase di copia dei dati dal file alla memoria allocata per il nemico, la battaglia viene comunque elaborata ma con dati errati e visualizzando diversi errori.	
Scenario di base	La lotta contro un nemico avviene senza alcun problema di sorta e termina o con la sua sconfitta o con la sconfitta del giocatore.	
Scenario alternativo	Visto che la Battaglia elabora diverse altre fasi potrebbero esserci diversi scenari alternativi, potrebbero esserci problemi con la gestione dei valori temporanei, con i dati allocati del nemico o problemi di calcolo non previsti che potrebbero anche portare ad un crash del gioco.	

Codice	Nome	Descrizione
RB14	Caricamento Battaglia	Legge la posizione del giocatore ed in base ad essa legge il file del nemico che si trova in tale posizione e lo carica in memoria e ne avvia la battaglia.
Pre-condizioni	Scelta di Lottare in una Stanza con la presenza di un nemico nascosto.	
Post-condizioni	Successo: viene utilizzato il valore di posizione del giocatore per sapere in quale stanza di trova, tramite questo valore viene inizializzata la memoria allocata per il nemico con i dati relativi al nemico presenti sul relativo file. Viene avviata la battaglia con musica. Fallimento: non vengono copiati i dati da file	
Scenario di base	La battaglia contro il nemico si svolge normalmente.	
Scenario alternativo	Nel caso di problemi con la copia dei dati da file, la battaglia avrà comunque luogo, ma si baserà sui dati precedentemente memorizzati nella memoria allocata per il nemico, lasciando a schermo comunque un messaggio di errore relativo alla mancata lettura/apertura del file. Nel caso di mancanza di file audio, il sistema riprodurrà un avviso sonoro.	

Codice	Nome	Descrizione
RGo1	Menu di Avvio	Menu di scelta iniziale per l'azione che si vuole svolgere nel gioco.
Pre-condizioni	Caricamento del programma	
Post-condizioni	Apertura del menù con le possibili scelte	
Scenario di base	Viene presentato, dopo l'immagine iniziale del gioco, un menù dove è possibile scegliere tra l'iniziare una nuova partita, caricarne una già esistente oppure entrare nella sezione admin	

Codice	Nome	Descrizione
RGo2	Descrizione della Stanza	Legge la posizione del giocatore ed in base ad essa stampa a schermo la relativa descrizione.
Pre-condizioni	Ritorno al menù di gioco dopo essersi spostati tramite la funzionalità di avanzamento.	
Post-condizioni	Successo: Viene stampata a schermo la descrizione corretta relativa alla stanza in cui il giocatore si trova in quel momento. Fallimento: Viene stampata una descrizione errata oppure non viene stampata alcuna descrizione.	
Scenario di base	Il giocatore ritornando al menù di gioco potrà leggere la descrizione della stanza nel quale si trova.	
Scenario alternativo	A seconda dell'errore causato da un malfunzionamento della funzione avanzamento, potrebbe esserci una descrizione errata oppure la funzione non riconoscendo la stanza in cui si trova non stampa alcuna descrizione.	

Codice	Nome	Descrizione
RGo3	Interfaccia Equipaggiamento	Legge i dati del giocatore e stampa a schermo l'interfaccia del menù Equipaggiamento mostrando solo gli oggetti posseduti.
Pre-condizioni	Scelta della voce "Equipaggiamento" dal menu di gioco.	
Post-condizioni	Successo: Viene stampata a schermo l'interfaccia dell'equipaggiamento con tutti gli oggetti posseduti visibili. Fallimento: Viene stampata una forma errata dell'interfaccia.	
Scenario di base	L'interfaccia visualizzata contiene i nomi in chiaro di tutti gli oggetti ottenuti in gioco.	
Scenario alternativo	Nel caso di problemi con gli eventi o durante qualche modifica in attesa dei campi relativi all'equipaggiamento, potrebbero visualizzarsi oggetti non ancora posseduti o non vedersi oggetti posseduti realmente.	

Codice	Nome	Descrizione
RGo4	Aggiunta Equipaggiamento	Legge da file i valori dell'equipaggiamento ed aggiorna le statistiche del giocatore.
Pre-condizioni	L'esistenza dei file inerenti ai vari equipaggiamenti e la scelta da parte del giocatore di indossarne uno.	
Post-condizioni	Successo: Il file viene letto correttamente, vengono estrapolate le informazioni da esso e aggiunte le varie statistiche al giocatore. Fallimento: Il fallimento è previsto solo per la mancata lettura del file, perché il controllo sulla possibilità di poter equipaggiare un oggetto viene effettuato da un'altra funzione.	
Scenario di base	Il giocatore non riceve alcuna informazione, una volta richiesto di indossare un equipaggiamento, potrà controllare i valori aggiornati nella tabella statistiche.	
Scenario alternativo	In caso di mancata lettura del file ci sarà un errore visualizzato a schermo.	

Codice	Nome	Descrizione
RGo5	Rimozione Equipaggiamento	Legge da file i valori dell'equipaggiamento ed aggiorna le statistiche del giocatore.
Pre-condizioni	L'esistenza dei file inerenti ai vari equipaggiamenti e la scelta da parte del giocatore di indossarne uno già indossato.	
Post-condizioni	Successo: Il file viene letto correttamente, vengono estrapolate le informazioni da esso e rimosse le varie statistiche dal giocatore. Fallimento: Il fallimento è previsto solo per la mancata lettura del file, perché il controllo sulla possibilità di poter equipaggiare un oggetto viene effettuato da un'altra funzione.	
Scenario di base	Il giocatore non riceve alcuna informazione, una volta richiesto di indossare un equipaggiamento, potrà controllare i valori aggiornati nella tabella statistiche.	
Scenario alternativo	In caso di mancata lettura del file ci sarà un errore visualizzato a schermo.	

Codice	Nome	Descrizione
RGo6	Menù Equipaggiamento	Elabora l'interfaccia, l'aggiunta e la rimozione dell'equipaggiamento e controlla che il giocatore non indossi più pezzi della stessa tipologia di equipaggiamento.
Pre-condizioni	Scelta dal menu di gioco della voce "Equipaggiamento"	
Post-condizioni	Successo: Il giocatore dopo aver visto l'equipaggiamento ottenuto durante tutto il gioco potrà scegliere quale indossare. Fallimento: Nel caso l'oggetto scelto è già indossato o vi è già indossato un oggetto della medesima categoria non viene permesso al giocatore di indossarne un altro.	
Scenario di base	Il giocatore sceglie cosa indossare, viene stampata a schermo una piccola descrizione e nel caso di conferma l'equipaggiamento scelto viene indossato e le statistiche aggiornate.	
Scenario alternativo	Nel caso in cui il giocatore indossi già un equipaggiamento di una determinata categoria, esempio un arma, verrà avvisato di indossarne già una e quindi invitato a selezionarla e rimuoverla prima di indossarne un'altra.	

Codice	Nome	Descrizione
RGo7	Menù Consumabili in Esplorazione	Utilizza la stessa elaborazione dei consumabili in battaglia senza utilizzare i dati temporanei.
Pre-condizioni	Scelta dal menù di gioco della voce "Consumabili"	
Post-condizioni	Successo: Appare l'interfaccia dei consumabili, il giocatore può scegliere un consumabile ed usarlo, il contatore dei consumabili viene aggiornato. Funziona in maniera analoga ai consumabili in battaglia ma con una conversione dei valori temporanei a quelli non temporanei, ciò non comporta alcun cambiamento a livello pratico ma fa da predisposizione per salvataggi in battaglia che si baserebbero sui soli dati non temporanei e quindi precedenti alla battaglia. Fallimento: In caso non ci sia disponibilità il giocatore viene avvisato della mancata quantità.	
Scenario di base	Il giocatore sceglie dalla lista il consumabile e si potenzia.	
Scenario alternativo	Il giocatore che dovesse scegliere un consumabile esaurito riceve un messaggio di errore avvisandolo della mancanza di tale oggetto.	

Codice	Nome	Descrizione
RGo8	Gestione delle Interazione	Legge la posizione del giocatore ed avvia l'eventuale evento legato alla sua posizione.
Pre-condizioni	Scelta della voce del menù di gioco "Interazione"	
Post-condizioni	Successo: Il programma controlla la stanza in cui si trova il giocatore, trova la corrispondenza con il relativo evento di gioco e viene avviata la relativa funzione. Fallimento: Il programma avvisa il giocatore che non c'è nulla con cui interagire.	
Scenario di base	Il giocatore interagisce con l'ambiente e quindi prosegue la sua avventura con le conseguenze della sua interazione.	
Scenario alternativo	Il giocatore una volta avvisato della mancata presenza di qualcosa con cui interagire, può proseguire con la sua avventura.	

Codice	Nome	Descrizione
RGo9	Inizializzazione dati Giocatore	Pone a valore neutro i dati allocati in memoria relativi al giocatore.
Pre-condizioni	Avvenuta allocazione delle strutture character ed enemy.	
Post-condizioni	Successo: Il programma pone a o i vari record numerici e "Player" nel record del nome giocatore.	
Scenario di base	Il giocatore non si accorge di nulla e quindi il gioco prosegue normalmente.	
Scenario alternativo	Nel caso di problemi potrebbe capitare un crash, mentre gli altri dati verrebbero cmq poi sovrascritti con l'inizio dell'avventura o con il caricamento.	

Codice	Nome	Descrizione
RG10	Scelta Nome Giocatore	Scrive in memoria il nome scelto dal giocatore per il proprio personaggio.
Pre-condizioni	Avvio di una nuova partita.	
Post-condizioni	Successo: Il programma chiede al giocatore di inserire il suo nome, chiede conferma e dopo lo copia nel record del nome giocatore della struttura character. Fallimento: Se rifiuta di accettare il nome scelto, potrà inserirne uno nuovo.	
Scenario di base	Il giocatore scrive il nome del suo protagonista e dopo averlo confermato proseguirà con la sua avventura.	
Scenario alternativo	Il giocatore potrà eventualmente rifiutare di confermare il nome scelto e avere la possibilità di inserirne uno nuovo.	

Codice	Nome	Descrizione
RG11	Selezione Classe Giocatore	Permette la scelta della Classe iniziale di gioco e scrive nei dati relativi alla classe scelta nella memoria allocata per il giocatore.
Pre-condizioni	Aver scelto il nome del proprio personaggio.	
Post-condizioni	Successo: Il programma dopo aver mostrato a schermo le varie immagini delle classi con le relative informazioni chiede di sceglierne una. Dopo la scelta il programma fa il confronto della stringa ricevuta per determinare quali valori assegnare alle varie statistiche. Fallimento: Nel caso in cui si sbaglia a scrivere, viene visualizzato un errore e data nuovamente la possibilità di scegliere la classe.	
Scenario di base	Il giocatore dopo la richiesta del gioco scrive la classe scelte ed il gioco procede.	
Scenario alternativo	Il giocatore sbaglia a scrivere e quindi ha la possibilità di riscrivere la classe da lui scelta.	

Codice	Nome	Descrizione
RG12	Imposta Posizione Iniziale	Scrive nella memoria allocata per il giocatore i dati sulla posizione iniziale.
Pre-condizioni	Aver scelto la classe iniziale di gioco.	
Post-condizioni	Successo: I record relativi alle coordinate vengono scritte con le coordinate iniziali della mappa delle catacombe.	
Scenario di base	Il giocatore non si accorge di nulla e si ritrova direttamente nell'atto 1	

Codice	Nome	Descrizione
RG13	Atto 0	Elabora l'introduzione del gioco, scelta della classe, nome giocatore e prepara i dati per il primo atto.
Pre-condizioni	Aver scelto di avviare una nuova partita.	
Post-condizioni	Successo: Il programma introduce l'ambientazione di gioco al giocatore e avvia le varie funzioni riguardanti la scelta del nome e della classe, scrive le coordinate delle catacombe e come ultima cosa il flag di superamento dell'Atto 0 viene posto ad 1.	
Scenario di base	Il giocatore viene introdotto al mondo di gioco, sceglie il suo nome, la classe e quindi inizia l'atto 1 nelle catacombe.	
Scenario alternativo	Potrebbe non portare a termine l'atto 0 per via di problematiche relative alle dipendenze della funzione. Leggere Informazioni ad inizio paragrafo.	

Codice	Nome	Descrizione
RG14	Stampa Mappa Catacombe	Legge la posizione del giocatore e in base ad essa renderizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
Pre-condizioni	Scelta della voce "Mappa" o "Avanza" durante l'atto1.	
Post-condizioni	Successo: Il programma in base alle coordinate, stamperà riga per riga la mappa ponendo la X nella posizione del giocatore. Fallimento: Se non c'è corrispondenza con le coordinate previste che il giocatore possa occupare, la stampa avviene senza la X ad indicare la posizione del giocatore.	
Scenario di base	Il giocatore vede a schermo la mappa e la sua posizione sulla mappa.	
Scenario alternativo	In caso di coordinate errate o fuori parametro, la mappa appare senza posizione.	

Codice	Nome	Descrizione
RG15	Controllo Mura Mappa Catacombe	Legge la posizione del giocatore e in base ad essa renderizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
Pre-condizioni	Il giocatore sceglie di avanzare in una nuova posizione all'interno della mappa.	
Post-condizioni	Successo: Il programma fa un confronto tra le coordinate che il giocatore vuole intraprendere con un array bidimensionale che contiene le celle dove il giocatore è libero di muoversi e ritorna un valore indicante la possibilità di muoversi in quella direzione. Fallimento: Dopo il confronto con l'array nel caso in cui il giocatore voglia spostarsi verso una zona bloccata da un muro la funzione ritorna un valore indicante l'impossibilità di procedere verso quella direzione.	
Scenario di base	Il giocatore non si accorge di tale funzione. Sarà il requisito funzionale riguardante l'avanzamento poi ad avvisare il giocatore se ci sono problemi con l'avanzamento verso la direzione scelta.	

Codice	Nome	Descrizione
RG16	Avanzamento Testuale Mappa Catacombe	Elabora il movimento del giocatore nella mappa tramite testo.
Pre-condizioni	Il giocatore sceglie di avanzare verso una nuova posizione dopo aver scelto inizialmente di utilizzare la funzione di avanzamento testuale.	
Post-condizioni	Successo: Il programma legge la direzione che il giocatore vuole intraprendere e modifica i parametri dello struct inerenti alle posizioni temporanee poi fa uso di RG15 che ritornerà un valore che permette l'avanzamento e le coordinate vengono aggiornate. Fallimento: Se RG15 da esito negativo, verrà dato un messaggio al giocatore.	
Scenario di base	Il giocatore vedrà muovere il cursore della sua posizione sulla mappa.	
Scenario alternativo	Il giocatore non vedrà muovere il cursore della sua posizione sulla mappa ed un messaggio lo avviserà che non può procedere verso tale direzione.	

Codice	Nome	Descrizione
RG17	Avanzamento con Freccie Mappa Catacombe	Elabora il movimento del giocatore nella mappa tramite l'uso delle frecce.
Pre-condizioni	Il giocatore sceglie di avanzare verso una nuova posizione dopo aver scelto inizialmente di utilizzare la funzione di avanzamento tramite frecce.	
Post-condizioni	Successo: Il programma legge la pressione del tasto della freccia e modifica i parametri dello struct inerenti alle posizioni temporanee poi fa uso di RG15 che ritornerà un valore che permette l'avanzamento e le coordinate vengono aggiornate. Fallimento: Se RG15 da esito negativo, verrà dato un messaggio al giocatore.	
Scenario di base	Il giocatore vedrà muovere il cursore della sua posizione sulla mappa.	
Scenario alternativo	Il giocatore non vedrà muovere il cursore della sua posizione sulla mappa ed un messaggio lo avviserà che non può procedere verso tale direzione. Inoltre, l'uso di tale funzione crea delle problematiche sul buffer della tastiera, quindi, comandi impartiti usando altri tasti durante lo spostamento potrebbero poi venire eseguiti al termine dello spostamento, creando scenari imprevedibili.	

Codice	Nome	Descrizione
RG18	Controllo Stanza Mappa Catacombe	Assegna a delle coordinate un carattere che identifica una stanza.
Pre-condizioni	Esplorazione della mappa dopo aver usato la funzionalità di avanzamento.	
Post-condizioni	Il giocatore non si accorge di tale funzionalità. Successo: Il programma prendendo le coordinate della mappa in cui si trova il giocatore le utilizzerà per accedere ad un array bidimensionale dimensioni uguali alla mappa contenente i caratteri relativi alle stanze.	
Scenario di base	Il giocatore tornando al menù di gioco dopo aver usato la funzionalità avanzamento leggerà le informazioni corrette inerenti alla stanza in cui si trova.	
Scenario alternativo	Il giocatore tornando al menù di gioco dopo aver usato la funzionalità avanzamento leggerà le informazioni sbagliate rispetto alla stanza in cui si trova.	

Codice	Nome	Descrizione
RG19	Atto 1	Elabora l'intero atto: il menù di gioco durante l'esplorazione della mappa con le relative funzionalità e la fine dell'atto dopo la sconfitta del boss locale.
Pre-condizioni	Aver terminato l'atto 0	
Post-condizioni	Successo: Il programma stampa a schermo il menù di gioco e facendo uso di altre funzionalità elabora tutte le scelte del giocatore, tra cui la visualizzazione della mappa, l'uso di consumabili, l'accesso al salvataggio, ecc. Fallimento: Non previsto poiché l'accesso avviene tramite flag.	
Scenario di base	Il giocatore ha accesso al menù di gioco e può usufruire di ogni comando a sua disposizione.	
Scenario alternativo	Se ci dovessero essere problemi con le flag dei dati giocatore come dati corrotti o modifica accidentale di tali flag, il giocatore potrebbe non accedere mai all'atto.	

Codice	Nome	Descrizione
RG20	Imposta Posizione Atto2	Scrive nella memoria allocata per il giocatore i dati sulla posizione iniziale per l'atto 2.
Pre-condizioni	Aver superato l'NPC dell'atto intermezzo.	
Post-condizioni	Successo: I record relativi alle coordinate vengono scritte con le coordinate iniziali della mappa del castello.	
Scenario di base	Il giocatore non si accorge di nulla e si ritrova direttamente nell'atto intermezzo	

Codice	Nome	Descrizione
RG21	Evento Intermezzo	Elabora l'incontro con un NPC che permetterà al giocatore l'avanzamento all'atto 2.
Pre-condizioni	Accesso all'atto intermezzo.	
Post-condizioni	Successo: Il programma elabora un incontro con un NPC memorizzando le scelte effettuate dal giocatore durante il loro dialogo, se verranno soddisfatte le condizioni per cui le scelte sono soddisfacenti per l'NPC, il programma modificherà i flag per permettere il proseguimento della storia dando l'accesso all'atto 2. Fallimento: Nel caso in cui le scelte non sono soddisfacenti per l'NPC, il programma riavvia l'incontro.	
Scenario di base	Il giocatore verrà introdotto in un dialogo ed interagisce tramite testo con l'NPC, una volta terminata la conversazione verrà introdotto verso una nuova parte della storia.	
Scenario alternativo	Il giocatore vedrà ricevere un rifiuto da parte dell'NPC e si ritroverà a dover nuovamente conversare con lui per avanzare.	

Codice	Nome	Descrizione
RG22	Atto Intermezzo	Elabora l'intermezzo del gioco, musica, testi, immagini dal post atto 1, l'evento intermezzo fino alla preparazione dei dati per l'atto 2.
Pre-condizioni	Aver sconfitto il boss delle catacombe.	
Post-condizioni	Successo: Il programma elabora tutto il necessario per usufruire di tale atto, tra cui musica, testo, immagini e l'evento cruciale di tale atto. Fallimento: Vedi RG19	
Scenario di base	Il giocatore si ritrova a vivere la storia che lo porterà alla prossima zona.	
Scenario alternativo	Vedi RG19	

Codice	Nome	Descrizione
RG23	Stampa Mappa Castello	Legge la posizione del giocatore e in base ad essa renderizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
Pre-condizioni	Scelta della voce "Mappa" o "Avanza" durante l'atto2.	
Post-condizioni	Successo: Il programma in base alle coordinate, stamperà riga per riga la mappa ponendo la X nella posizione del giocatore. Fallimento: Se non c'è corrispondenza con le coordinate previste che il giocatore possa occupare, la stampa avviene senza la X ad indicare la posizione del giocatore.	
Scenario di base	Il giocatore vede a schermo la mappa e la sua posizione sulla mappa.	
Scenario alternativo	In caso di coordinate errate o fuori parametro, la mappa appare senza posizione.	

Codice	Nome	Descrizione
RG24	Controllo Mura Mappa Castello	Legge la posizione del giocatore e in base ad essa renderizza l'intera mappa posizionando una "X" nel luogo in cui si trova.
Pre-condizioni	Il giocatore sceglie di avanzare in una nuova posizione all'interno della mappa.	
Post-condizioni	Successo: Il programma fa un confronto tra le coordinate che il giocatore vuole intraprendere con un array bidimensionale che contiene le celle dove il giocatore è libero di muoversi e ritorna un valore indicante la possibilità di muoversi in quella direzione. Fallimento: Dopo il confronto con l'array nel caso in cui il giocatore voglia spostarsi verso una zona bloccata da un muro la funzione ritorna un valore indicante l'impossibilità di procedere verso quella direzione.	
Scenario di base	Il giocatore non si accorge di tale funzione. Sarà il requisito funzionale riguardante l'avanzamento poi ad avvisare il giocatore se ci sono problemi con l'avanzamento verso la direzione scelta.	

Codice	Nome	Descrizione
RG25	Avanzamento Testuale Mappa Castello	Elabora il movimento del giocatore nella mappa tramite testo.
Pre-condizioni	Il giocatore sceglie di avanzare verso una nuova posizione dopo aver scelto inizialmente di utilizzare la funzione di avanzamento testuale.	
Post-condizioni	Successo: Il programma legge la direzione che il giocatore vuole intraprendere e modifica i parametri dello struct inerenti alle posizioni temporanee poi fa uso di RG24 che ritornerà un valore che permette l'avanzamento e le coordinate vengono aggiornate. Fallimento: Se RG24 da esito negativo, verrà dato un messaggio al giocatore.	
Scenario di base	Il giocatore vedrà muovere il cursore della sua posizione sulla mappa.	
Scenario alternativo	Il giocatore non vedrà muovere il cursore della sua posizione sulla mappa ed un messaggio lo avviserà che non può procedere verso tale direzione.	

Codice	Nome	Descrizione
RG26	Avanzamento con Freccie Mappa Catacombe	Elabora il movimento del giocatore nella mappa tramite l'uso delle frecce.
Pre-condizioni	Il giocatore sceglie di avanzare verso una nuova posizione dopo aver scelto inizialmente di utilizzare la funzione di avanzamento tramite frecce.	
Post-condizioni	Successo: Il programma legge la pressione del tasto della freccia e modifica i parametri dello struct inerenti alle posizioni temporanee poi fa uso di RG24 che ritornerà un valore che permette l'avanzamento e le coordinate vengono aggiornate. Fallimento: Se RG24 da esito negativo, verrà dato un messaggio al giocatore.	
Scenario di base	Il giocatore vedrà muovere il cursore della sua posizione sulla mappa.	
Scenario alternativo	Il giocatore non vedrà muovere il cursore della sua posizione sulla mappa ed un messaggio lo avviserà che non può procedere verso tale direzione. Inoltre, l'uso di tale funzione crea delle problematiche sul buffer della tastiera, quindi, comandi impartiti usando altri tasti durante lo spostamento potrebbero poi venire eseguiti al termine dello spostamento, creando scenari imprevedibili.	

Codice	Nome	Descrizione
RG27	Controllo Stanza Mappa Castello	Assegna a delle coordinate un carattere che identifica una stanza.
Pre-condizioni	Esplorazione della mappa dopo aver usato la funzionalità di avanzamento.	
Post-condizioni	Il giocatore non si accorge di tale funzionalità. Successo: Il programma prendendo le coordinate della mappa in cui si trova il giocatore le utilizzerà per accedere ad un array bidimensionale dimensioni uguali alla mappa contenente i caratteri relativi alle stanze.	
Scenario di base	Il giocatore tornando al menù di gioco dopo aver usato la funzionalità avanzamento leggerà le informazioni corrette inerenti alla stanza in cui si trova.	
Scenario alternativo	Il giocatore tornando al menù di gioco dopo aver usato la funzionalità avanzamento leggerà le informazioni sbagliate rispetto alla stanza in cui si trova.	

Codice	Nome	Descrizione
RG28	Atto 2	Elabora l'intero atto: il menù di gioco durante l'esplorazione della mappa con le relative funzionalità e la fine dell'atto dopo la sconfitta del boss locale.
Pre-condizioni	Aver terminato l'atto intermezzo	
Post-condizioni	Successo: Il programma stampa a schermo il menù di gioco e facendo uso di altre funzionalità elabora tutte le scelte del giocatore, tra cui la visualizzazione della mappa, l'uso di consumabili, l'accesso al salvataggio, ecc. Fallimento: Non previsto poiché l'accesso avviene tramite flag.	
Scenario di base	Il giocatore ha accesso al menù di gioco e può usufruire di ogni comando a sua disposizione.	
Scenario alternativo	Se ci dovessero essere problemi con le flag dei dati giocatore come dati corrotti o modifica accidentale di tali flag, il giocatore potrebbe non accedere mai all'atto.	

Codice	Nome	Descrizione
RG29	Atto Epilogo	Elaborazione del testo, immagini, musica e crediti dell'epilogo di gioco.
Pre-condizioni	Aver sconfitto il boss dell'atto 2	
Post-condizioni	Successo: Il programma elabora ogni componente di tale atto, testi, immagini e musica. Fallimento: Vedi RG28	
Scenario di base	Il giocatore si ritrova a scoprire come termina la sua avventura e leggerà i credits di gioco.	
Scenario alternativo	Vedi RG28	

Codice	Nome	Descrizione
RI01-10	Elaborazione Casse	Elabora l'interazione del giocatore con una delle dieci casse presenti nel gioco.
Pre-condizioni	Avvio della funzionalità RGo8 trovandosi in una stanza contenente una cassa.	
Post-condizioni	Successo: Il programma elabora ogni cassa come funzioni singole, esse hanno contenuti unici e dopo aver aggiornato l'inventario del giocatore con i nuovi oggetti ottenuti modificano il flag relativo alla cassa aperta. Fallimento: Il programma controlla che una determinata cassa sia stata aperta tramite flag, in tal caso il programma non darà nuovamente accesso a tale cassa.	
Scenario di base	Il giocatore che si appresta ad aprire per la prima volta una cassa vedrà a schermo gli oggetti che vi erano al suo interno e li prenderà con sé.	
Scenario alternativo	Il giocatore ha già aperto la cassa, quindi, leggerà a schermo che la cassa è stata già aperta precedentemente.	

Codice	Nome	Descrizione
RI11-20	Elaborazione Eventi	Elabora l'interazione del giocatore con uno dei dieci eventi presenti nel gioco.
Pre-condizioni	Avvio della funzionalità RGo8 trovandosi in una stanza che da avvio ad un evento.	
Post-condizioni	Successo: Il programma elabora ogni evento come singole funzioni. In linea generale un evento è strutturato in una parte testuale introduttiva, un'interazione (non sempre necessaria) del giocatore e l'elaborazione di un eventuale loot e relativo flag. Fallimento: Il programma controlla che un determinato evento sia già avvenuto tramite flag, in tal caso il programma non darà nuovamente accesso a tale evento.	
Scenario di base	Il giocatore affronterà l'evento.	
Scenario alternativo	Il giocatore riceverà in base all'evento già avvenuto un messaggio o interazione per cui non darà seguito nuovamente all'evento.	

Codice	Nome	Descrizione
RA01	Menu Opzioni	Elabora la schermata admin e il suo relativo menù.
Pre-condizioni	Il menù deve essere scelto tra le opzioni presenti nel menu principale.	
Post-condizioni	Il menù amministratore viene aperto.	
Scenario di base	Viene stampato a schermo il menù con le sue possibili opzioni.	

Codice	Nome	Descrizione
RA02	Inserimento password	Legge la password inserita dall'utente.
Pre-condizioni	La funzione viene attivata solamente durante la registrazione dell'utente nell'area amministrativa.	
Post-condizioni	La funzione, nel caso viene chiamata per le condizioni precedenti, si avvia	
Scenario di base	Si apre una tabella con le condizioni per la quale la password, inserita dall'utente, possa essere accettata.	

Codice	Nome	Descrizione
RA03	Controllo numeri e maiuscole	Controlla se la password inserita dall'utente contiene i vari parametri richiesti.
Pre-condizioni	La funzione viene attivata solamente all'interno di RA02.	
Post-condizioni	La funzione controlla se la password rispetta le condizioni richieste	
Scenario di base	Si apre la funzione controllando se la password inserita dall'utente è corretta o meno	

Codice	Nome	Descrizione
RA04	Autenticazione utente.	Legge dal file se l'username e la password inseriti dall'utente coincidono con il file.
Pre-condizioni	La funzione viene attivata solamente durante l'accesso dell'utente nell'area amministrativa.	
Post-condizioni	Viene richiesto il nome utente e la password.	
Scenario di base	Nel caso in cui nome utente e password coincidano con un utente scritto nel file, allora l'utente entrerà nella zona cheat.	
Scenario alternativo	Nel caso in cui nome utente e password non coincidano con un utente scritto nel file, allora verranno richiesti i dati per un massimo di tre volte. Dopo queste tre, l'utente viene rimandato nel menu principale della sezione admin.	

Codice	Nome	Descrizione
RA05	Salvataggio dei dati.	Carica i dati dell'utente all'interno del file.
Pre-condizioni	La funzione può essere chiamata in qualsiasi momento del gioco, oppure per salvare i dati modificati nella sezione admin.	
Post-condizioni	Si attiva la funzione per il salvataggio dei dati.	
Scenario di base	Si presenta la richiesta di salvare i dati. In caso di sì, si presenteranno gli slot di salvataggio.	
Scenario alternativo	Si presenta la richiesta di salvare i dati. In caso di no, si ritorna da dove si è richiesto il salvataggio.	

Codice	Nome	Descrizione
RAo6	Caricamento dei dati.	Preleva dal file i dati dell'utente.
Pre-condizioni	La funzione può essere chiamata in qualsiasi momento del gioco, oppure per caricare i dati nella sezione admin.	
Post-condizioni	Vengono caricati i dati di uno slot all'interno del gioco o all'interno della zona cheat del menù amministratore.	
Scenario di base	Una volta chiamata, si presenta un menù dove poter scegliere tra i tre slot di salvataggio.	

Codice	Nome	Descrizione
RAo7	Salvataggio nascosto dei dati.	Carica direttamente i dati dell'utente all'interno del file, senza i passaggi dai vari menù e richieste.
Pre-condizioni	La funzione viene richiamata solamente dopo la modifica dei dati nella sezione admin.	
Post-condizioni	La funzione salva i dati modificati nello slot salvataggio precedentemente scelto.	
Scenario di base	La funzione viene chiamata e, tramite l'utilizzo di un flag precedentemente dichiarato, salva all'interno dello slot salvataggio scelto all'inizio dall'utente	

Codice	Nome	Descrizione
RAo8	Menu modifiche dati.	Elabora la schermata per la scelta dei dati da modificare.
Pre-condizioni	Funzione che viene attivata solamente dopo l'accesso riuscito della sezione admin.	
Post-condizioni	Vengono presentate le possibili scelte per le modifiche dei dati.	
Scenario di base	Si presenta un menù dove si può scegliere se modificare le statistiche, i consumabili, gli equipaggiamenti oppure uscire dal menù	

Codice	Nome	Descrizione
RA09	Modifiche delle statistiche.	Modifica le statistiche del personaggio in base alla scelta dell'amministratore.
Pre-condizioni	La funzione si avvia quando viene scelto di modificare le statistiche del personaggio.	
Post-condizioni	Si possono modificare le statistiche del personaggio in base alle proprie preferenze.	
Scenario di base	Si presenta un menù (Vedi RA12) dove vengono rappresentate le statistiche correnti del personaggio. L'utente può allora scegliere cosa modificare e, nel caso in cui ha finito, di uscire da questo menu.	

Codice	Nome	Descrizione
RA10	Modifiche dei consumabili.	Modifica i consumabili del personaggio in base alla scelta dell'amministratore.
Pre-condizioni	La funzione si avvia quando viene scelto di modificare i consumabili del personaggio.	
Post-condizioni	Si possono modificare i consumabili del personaggio in base alle proprie preferenze.	
Scenario di base	Si presenta un menù (Vedi RA13) dove vengono rappresentati i consumabili correnti del personaggio. L'utente può allora scegliere cosa modificare e, nel caso in cui ha finito, di uscire da questo menu.	

Codice	Nome	Descrizione
RA11	Modifiche degli equipaggiamenti.	Modifica gli equipaggiamenti del personaggio. L'amministratore può decidere se rendere disponibili la/le arma/i oppure no.
Pre-condizioni	La funzione si avvia quando viene scelto di modificare l'equipaggiamento del personaggio.	
Post-condizioni	Si possono modificare gli equipaggiamenti del personaggio in base alle proprie preferenze.	
Scenario di base	Si presenta un menù (Vedi RGo3) dove vengono rappresentati gli equipaggiamenti correnti del personaggio. L'utente può allora scegliere se rendere disponibile l'equipaggiamento oppure no oppure uscire dal menu una volta apportate le modifiche.	

Codice	Nome	Descrizione
RA12	Tabella per le statistiche.	Tabella per la rappresentazione delle statistiche.
Pre-condizioni	Scelta della voce "Statistiche" dal menu delle modifiche del giocatore.	
Post-condizioni	Viene stampata a schermo l'interfaccia delle statistiche con tutti gli oggetti posseduti visibili.	
Scenario di base	L'interfaccia visualizzata contiene i nomi di tutti gli oggetti disponibili ottenuti in gioco.	

Codice	Nome	Descrizione
RA13	Tabella per i consumabili.	Tabella per la rappresentazione dei consumabili.
Pre-condizioni	Scelta della voce "Consumabili" dal menu delle modifiche del giocatore.	
Post-condizioni	Viene stampata a schermo l'interfaccia dei consumabili con tutti gli oggetti posseduti visibili.	
Scenario di base	L'interfaccia visualizzata contiene i nomi di tutti gli oggetti disponibili ottenuti in gioco.	

2. Progettazione

2.1 Costanti

Nome Costante e Valore:

`NUM_ROWS_MAP1` 13

Descrizione: Indica la lunghezza della mappa 1

Posizione: game.h

Nome Costante e Valore:

`NUM_COLUMNS_MAP1` 20

Descrizione: Indica la larghezza della mappa 1

Posizione: game.h

Nome Costante e Valore:

`NUM_ROWS_MAP2` 15

Descrizione: Indica la lunghezza della mappa 2

Posizione: game.h

Nome Costante e Valore:

`NUM_COLUMNS_MAP2` 25

Descrizione: Indica la larghezza della mappa 2

Posizione: game.h

Nome Costante e Valore:

`max_username_lenght` 50

Descrizione: Massima lunghezza per l'username

Posizione: admin.h

Nome Costante e Valore:

`min_password_lenght` 8

Descrizione: Lunghezza minima della password

Posizione: admin.h

Nome Costante e Valore:

`max_password_lenght` 20

Descrizione: Lunghezza massima della password

Posizione: admin.h

Nome Costante e Valore:

`max_attempts` 3

Descrizione: Tentativi massimi per il login amministrativo

Posizione: admin.h

```
static const char walls_map1[NUM_ROWS_MAP1][NUM_COLUMNS_MAP1 + 1] {
    {"#####"}, //0
    {"#####"}, //1
    {"#####@###"}, //2
    {"#####@###@###"}, //3
    {"#####@###@###"}, //4
    {"##@###@###@###"}, //5
    {"##@###@###@###"}, //6
    {"#@@@@@@@@@@@@@@@@"}, //7
    {"##@###@#####@###"}, //8
    {"##@###@#####@###"}, //9
    {"##@###@###@###@###"}, //10
    {"#####@###"}, //11
    {"#####"}, //12
}
```

Descrizione: Array bidimensionale contenente le informazioni sulle mura della mappa 1
Posizione: game.h

```
static const char rooms_map1[NUM_ROWS_MAP1][NUM_COLUMNS_MAP1 + 1]{
    {"#####"},//0
    {"#####"},//1
    {"#####e@@@#"},//2
    {"#####i@@@@@h###d#@#"},//3
    {"#####@##@##f#c#b#@#"},//4
    {"##s####@##@#g@@@a#@#"},//5
    {"##@#t##@##@##@#@#@#"},//6
    {"#r@@@@@@@@@@@@@@@@@#"},//7
    {"##@#####@###@#@#"},//8
    {"##@p#####@#@#k#@#"},//9
    {"##q####o###nm##l###@#"},//10
    {"#####@#@#"},//11
    {"#####"},//12
}
```

Descrizione: Array bidimensionale contenente la posizione delle stanze nella mappa 1
Posizione: game.h

```
static const char walls_map2[NUM_ROWS_MAP2][NUM_COLUMNS_MAP2 + 1]{
{"#####"},//0
{"###@###@###@###@###@###"},//1
{"###@###@###@###@###@###"},//2
{"###@###@###@###@###@###"},//3
{"###@###@###@###@###@###"},//4
{"###@###@###@###@###@###"},//5
{"###@###@###@###@###@###"},//6
{"###@###@###@###@###@###"},//7
{"###@###@###@###@###@###"},//8
{"#####"},//9
{"#####"},//10
{"#####"},//11
{"#####"},//12
{"#####"},//13
{"#####"},//14
}
```

Descrizione: Array bidimensionale contenente le informazioni sulle mura della mappa 2
Posizione: game.h

Nome Costante:

```
static const char rooms_map2[NUM_ROWS_MAP2][NUM_COLUMNS_MAP2 + 1]{
    {"#####"},//0
    {"###a##b##c####@#####"},//1
    {"##a@@@@@@@@@#d#@###ee#"},//2
    {"###@##n##@##@#@###@#eeee#"},//3
    {"##f@f###hhh#@#####@eeee#"},//4
    {"###@##@#hhh#@###@###eeee#"},//5
    {"##g@@@@#####@###mmm###ee#"},//6
    {"###@###@#ll@@@@mmm@#####"},//7
    {"###@@##@#ll#@###mmm#@@p##"},//8
    {"#@@@@@#ll#@###@###@#####"},//9
    {"#@@@@@#ll@@@@###@###@c##"},//10
    {"###@@##@#####@#f#@#f#@#####"},//11
    {"###@##a@a####@@@@@@@@#####"},//12
    {"###@###o#####r#g#q#####"},//13
    {"#####"},//14
};
```

Descrizione: Array bidimensionale contenente la posizione delle stanze nella mappa 2

Posizione: game.h

2.2 Strutture Dati

Nome	Descrizione	Campi
enemy	Contiene tutti i campi per le informazioni relative ai nemici.	<pre> char enemy_name[40]; // nome nemico int hp; // salute int def; // difesa int m_def; // difesa magica int atk; // attacco int arc; // arcano int spd; // velocità char description[300]; // descrizione nemico char file_artwork[20]; // nome file immagine </pre>
character	Contiene tutti i campi per le informazioni relative al giocatore.	<pre> char player_name[50]; // nome giocatore int score; // punteggio int intro; // flag per intro int hp; // salute int def; // difesa int temp_def; // difesa temporanea int m_def; // difesa magica int temp_m_def; // difesa magica temporanea int atk; // attacco int temp_atk; // attacco temporaneo int arc; // arcano int temp_arc; // arcano temporaneo int spd; // velocità int temp_spd; // velocità temporanea int map; // mappa in cui ci si trova int x; // coordinata x mappa int old_x; // coordinata x precedente int y; // coordinata y mappa int old_y; // coordinata y precedente int arrows; // scelta tipo movimento struct consumables { // sottostruttura cons. int hp_potions; // Ampolla di Lacrime A. int def_potions; // Carne Essiccata I. int m_def_potions; // Scheggia di Luce Stellare int arc_potions; // Elisir Lunare int atk_potions; // Bolo Stimolante int spd_potions; // Piuma di Gufo // versioni dei consumabili utili per il reset int temp_hp_potions; int temp_def_potions; int temp_m_def_potions; int temp_arc_potions; int temp_atk_potions; int temp_spd_potions; } player_consumables; // nome sottostruttura struct events { // sottostruttura eventi int boss1; // flag boss catacom. int boss2; // flag boss castello int event_belethor; // flag evento inter. </pre>

		<pre> // elenco flag per gli eventi int event0; int event1; int event2; int event3; int event4; int event5; int event6; int event7; int event8; int event9; // elenco flag per le casse int chest0; int chest1; int chest2; int chest3; int chest4; int chest5; int chest6; int chest7; int chest8; int chest9; } player_events; // nome sottostruttura struct equip { // sottostruttura equipaggiamento int item0; // Shotel dell'Eclissi int item1; // Lama della Luna Crescente int item2; // Aculeo del Ghermitore int item3; // Martello Abissale int item4; // Falce della Dea Lunare int item5; // Oplon delle Maree int item6; // Scudospecchio Argenteo int item7; // Scudo della Progenie N. int item8; // Completo della C. N. int item9; // Veste dell'Ordine Lunare } player_equip; // nome sottostruttura int weapon; // indica l'uso di un'arma int shield; // indica l'uso di uno scudo int dress; // indica l'uso di un'armatura </pre>
administrator	Contiene l'id e la password dell'utente	<pre> char id[20]; char password[max_password_lenght + 1]; </pre>

2.3 File

Il programma utilizza quattro tipologie di file:

2.3.1 File di Salvataggio

I file di salvataggio sono file di scrittura/lettura di testo contenente informazioni di vario genere. Sono utilizzate per contenere i dati utente per il login nell'area amministrativa, dati di gioco degli utenti. Sono file formattati come csv, utilizzando ';' come separatore.

- login.txt
- data_player1.txt
- data_player2.txt
- data_player3.txt

2.3.2 File Audio

I file audio sono utilizzati dalla funzione `PlaySound()` (vedi approfondimento nella sezione Codifica) e sono di tipo .wav non compresso.

Qui un elenco di tali file:

- battle1.wav
- battle2.wav
- belethor.wav
- castle.wav
- catacombs.wav
- chest.wav
- door.wav
- epilogue1.wav
- exit.wav
- fontana.wav
- galuf.wav
- game_over.wav
- intro.wav
- love.wav
- moon.wav
- moon_reprise.wav
- moonlight.wav
- ost.wav
- rydia_death.wav
- rydia_encounter.wav
- selection.wav
- seluvis.wav

2.3.2 File Dati

I file dati sono file di testo contenente informazioni di vario genere. Sono utilizzate per contenere i dati dei vari nemici, statistiche dei vari equipaggiamenti o stringhe. Sono file formattati come csv, utilizzando ';' come separatore.

Qui l'elenco di tali file:

- aculeo.txt
- anagramma.txt
- armatura.txt
- completo.txt
- elementale
- eqp.txt
- falce.txt
- fantasma.txt
- galuf.txt
- indovinello.txt
- lama.txt
- martello.txt
- oplon.txt
- pipistrello.txt
- ragno.txt
- scheletro.txt
- scudo.txt
- scudospecchio.txt
- selene_d1_1.txt
- selene_d1_2.txt
- selene_d2_1.txt
- selene_d2_2.txt
- seluvis.txt
- shotel.txt
- veste.txt

2.3.3 File ASCII art

I file ASCII art sono file di testo contenenti caratteri ASCII per ricreare immagini.

Qui l'elenco di tali file:

- armor_ascii.txt
- act1menu.txt
- act2menu.txt
- bed.txt
- bat_ascii.txt
- cassaforte.txt
- door.txt
- cele.txt
- chest.txt
- classi.txt
- elemental_ascii.txt
- epilogue1.txt
- evento3.txt
- evento4.txt
- exit.txt
- game_over.txt
- galuf_ascii.txt
- ghost_ascii.txt
- line.txt
- name.txt
- lower_equipment.txt
- lower_equipment1.txt
- lower_main_menu.txt
- menu.txt
- menu_admin.txt
- menu_mod.txt
- seluvis_ascii.txt
- skeleton_ascii.txt
- spider_ascii.txt
- sky.txt
- title.txt
- title1.txt
- title2.txt
- tomba_seluvis.txt
- upper_equipment.txt
- upper_main_menu.txt
- wizard.txt

2.4 Librerie e Funzioni

Questo è l'elenco delle librerie utilizzate e di quelle create con le relative funzioni descritte.

LIBRERIE C

- `stdlib.h`
- `time.h`
- `stdio.h`
- `windows.h`
- `string.h`
- `conio.h`

LIBRERIE C++

- `iostream`
- `string`
- `fstream`
- `limits`

LIBRERIE CREATE

utility.h

Header che raccoglie varie funzioni utili all'elaborazione di testi ed immagini.

Funzione per la stampa di testo carattere per carattere con delay

- `void print_delay(char* text, int delay);`

Render immagine da file

- `void render_txt(char* text);`

Stampa di un testo in righe da cento caratteri.

- `void print_wrapped_text(char* text);`

battle.h

Header che raccoglie tutte le funzioni utili al combat system.

FUNZIONI DI UTILITA' AL FIGHT

Lettura da file dei dati del nemico e li mette in memoria

- `int enemy_initialize(char* enemyfilename, enemy* enemy);`

Prepara i valori temporanei per il fight

- `void tempstats_initialize(character* player);`

FUNZIONI INTERFACCIA

Menù Battaglia - Interfaccia

- `void fight_menu_interface(void);`

Tabella Statistiche

- `void tab_stats(character* player);`

Consumabili - Interfaccia

- `void f_consumables_interface(character* player);`

FUNZIONI PER ELABORAZIONE INVENTARIO IN LOTTA

Elaborazione Consumabili

- `void f_consumables(character* player);`

Aggiornamento dei Consumabili dai valori temporanei

- `void confirm_consumables(character* player);`

Sistema Loot/Punteggio post Lotta

- `void f_lootscore(character* player, int turn);`

FUNZIONI CALCOLO DANNI

Calcolo Danni Giocatore vs Nemico

- `int PvE_dmg_calculator(character* player, enemy* enemy);`

Calcolo Danni Nemico vs Giocatore

- `int EvP_dmg_calculator(enemy* enemy, character* player);`

Calcolo Danno in Difesa

- `void defence(character* player, enemy* enemy);`

ELABORAZIONE LOTTA

Elaborazione Turno (Colpo Critico, Priorità, Danno)

- `void attack(character* player, enemy* enemy);`

Elaborazione Menù Lotta

- `int battle(character* player, enemy* enemy);`

Caricamento dei dati dei Nemici in base alla posizione del Giocatore

- `int fight(character* player, enemy* enemy);`

interactions.h

Header contenente le funzioni sulle interazioni giocatore-ambiente

EVENTI E CHEST MAPPA 1

Evento: Tomba di Seluvis

- `void event0(character* player);`

Evento: Incontro con Selene

- `void event1(character* player);`

Evento: Fontana curativa

- `void event2(character* player);`

Evento: Indovinello

- `void event3(character* player);`

Evento: Decifra il messaggio

- `void event4(character* player);`

Chest Mappa 1 stanza s

- `void eventchest0(character* player);`

Chest Mappa 1 stanza o

- `void eventchest1(character* player);`

Chest Mappa 1 stanza l

- `void eventchest2(character* player);`

Chest Mappa 1 stanza d

- `void eventchest3(character* player);`

EVENTI E CHEST MAPPA 2

Evento: Pozione

- `void event5(character* player);`

Evento: Cassaforte con codice (5619543 dall'incontro con Selene)

- `void event6(character* player);`

Evento: Incontro con Lady Rydia

- `void event7(character* player);`

Evento: Biblioteca

- `void event8(character* player);`

Evento: Salone con il quadro di Celes

- `void event9(character* player);`

Chest Mappa 2

- `void eventchest4(character* player);`

Chest Mappa 2

- `void eventchest5(character* player);`

Chest Mappa 2

- `void eventchest6(character* player);`

Chest Mappa 2

- `void eventchest7(character* player);`

Chest Mappa 2

- `void eventchest8(character* player);`

Chest Mappa 2

- `void eventchest9(character* player);`

game.h

Header contenenti tutti gli elementi utili ad elaborare le funzionalità del gioco.

Elaborazione Avvio e Menu di Avvio

- `void title(character*player);`

FUNZIONI UTILI PER L'INTERO GIOCO

Testo Introduttivo ad ogni stanza

- `void introduction_room(character* player);`

Render menu equipaggiamento

- `void render_equipment(character* player);`

Legge da File Equip e li assegna

- `void add_eqpstats(char* fileeqp, character* player);`

Legge da File Equip e li rimuove

- `void rem_eqpstats(char* fileeqp, character* player);`

Elaborazione Menù Equipaggiamento

- `void equipment(character* player1);`

Menù Consumabili del Menu Principale

- `void map_consumables(character* player1);`

Elaborazione menù Interagisci

- `void interaction(character* player);`

FUNZIONI ATTO 0

Inizializza tutti a 0 tutti i valori di Player

- `void player_initialize(character* player);`

Scelta del nome

- `void name_selection(character* player);`

Selezione Classe

- `void class_selection(character* player);`

Imposta la Posizione del Giocatore in una nuova partita

- `void start_player_position(character* player);`

Elabora l'intero Atto 0 - Introduzione

- `void act0(character* player);`

FUNZIONI ATTO 1

Renderizza la mappa delle catacombe con la posizione del Giocatore

- `void render_map1(character* player);`

Controllo delle mura per l'avanzamento

- `int check_walls_map1_print(character* player, int temp_x, int temp_y);`

Voce Menù "Avanza" tramite testo

- `int advance_map1_text(character* player);`

Voce Menù "Avanza" tramite frecce

- `int advance_map1_arrows(character* player);`

Prende la posizione e restituisce l'id della stanza corrispondente

- `char positionTOrroom_map1(character* player);`

Elabora l'intero Atto 1 - Le Catacombe

- `void act1(character* player, enemy* enemy);`

FUNZIONI ATTO INTERMEZZO

Imposta la Posizione del Giocatore prima di iniziare l'Atto 2

- `void map2_player_position(character* player);`

Incontro con Belethor

- `void event_belethor(character* player);`

Elabora l'intero Atto - Intermezzo

- `void act_interlude(character* player);`

FUNZIONI ATTO 2

Renderizza la mappa delle catacombe con la posizione del Giocatore

- `void render_map2(character* player);`

Controllo delle mura per l'avanzamento

- `int check_walls_map2_print(character* player, int temp_x, int temp_y);`

Voce Menù "Avanza" tramite testo

- `int advance_map2_text(character* player);`

Voce Menù "Avanza" tramite frecce

- `int advance_map2_arrows(character* player);`

Prende la posizione e restituisce l'id della stanza corrispondente

- `char positionTOrroom_map2(character* player);`

Elabora l'intero Atto 2 - Il Castello

- `void act2(character* player, enemy* enemy);`

Elabora l'intero Atto Epilogo

`void act_epilogue();`

admin.h

Header contenenti tutti gli elementi utili per amministrare il gioco e contenere i salvataggi degli stessi.

Sezione admin per registrazione o modifica dei dati

- `void admin(character* player);`

Funzione che reinderizza il giocatore ai vari sottomenu

- `int menu(character* player);`

FUNZIONI REGISTRAZIONE

Funzione per inserire la password

- `void insert_password(char password);`

Controllo password per caratteri maiuscoli

- `int has_uppercase_number(char str);`

FUNZIONE LOGIN

Controllo utente per il login nell'area admin

- `int authenticate_user(const char* username, const char* password);`

FUNZIONI UTILI NELL'INTERO GIOCO

Funzione per il salvataggio dei dati utente

- `void saving(character* player);`

Funzione per il caricamento dei dati utente

- `int loading(character* player);`

FUNZIONI UTILI NELLA MODIFICA DEI DATI

Scrittura diretta dei dati sul file

- `void writing(character* player, int slot);`

Funzione per la modifica dei dati utente

- `void modify(character* player);`

Funzione per la modifica delle statistiche dell'utente

- `void modify_stats(character* player);`

Funzione per la modifica dei consumabili dell'utente

- `void modify_cons(character* player);`

Funzione per la modifica dell'equipaggiamento dell'utente

- `void modify_equip(character* player);`

FUNZIONI UTILI PER FINI RAPPRESENTATIVI

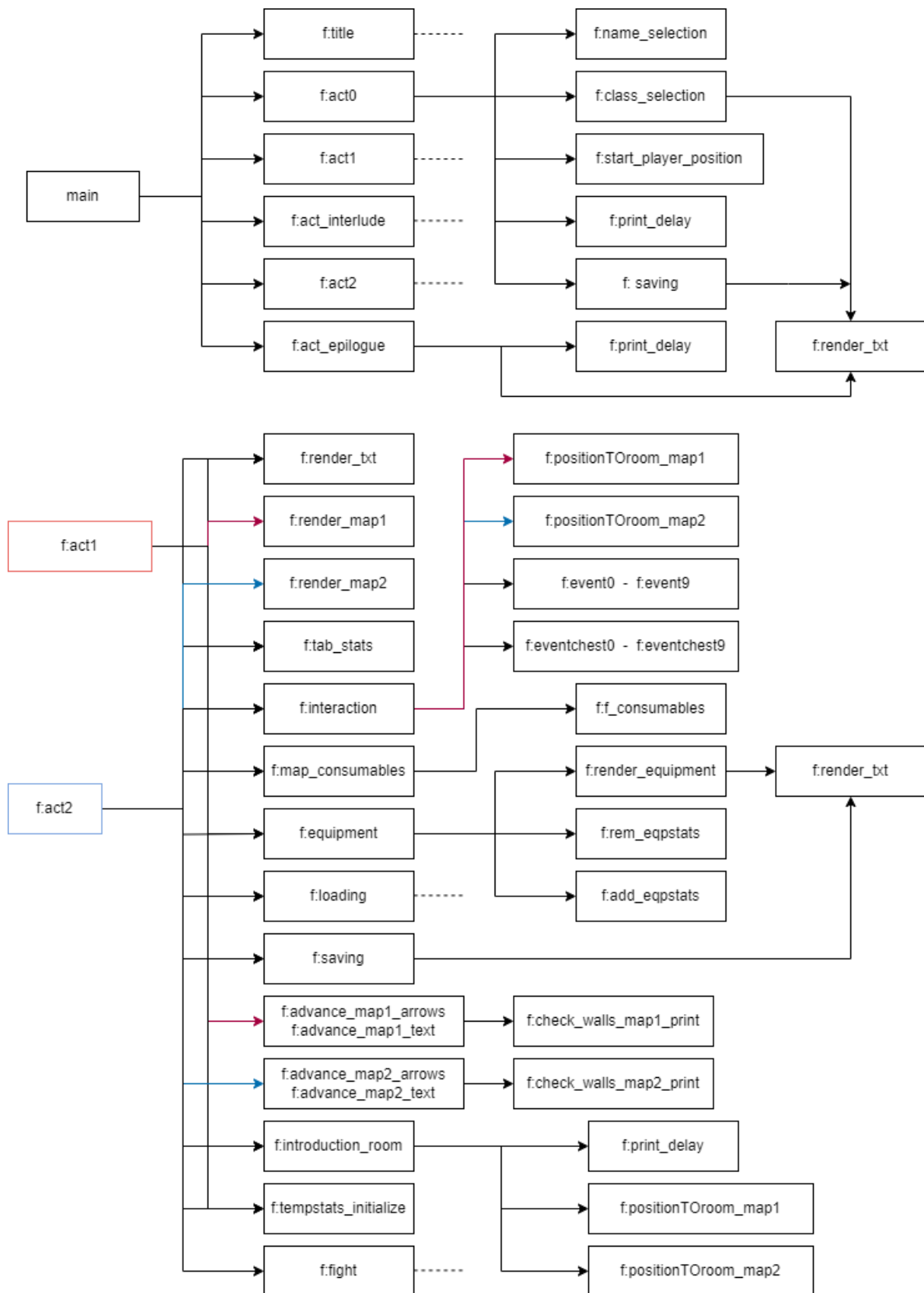
Funzione per mostrare la tabella dei valori del player

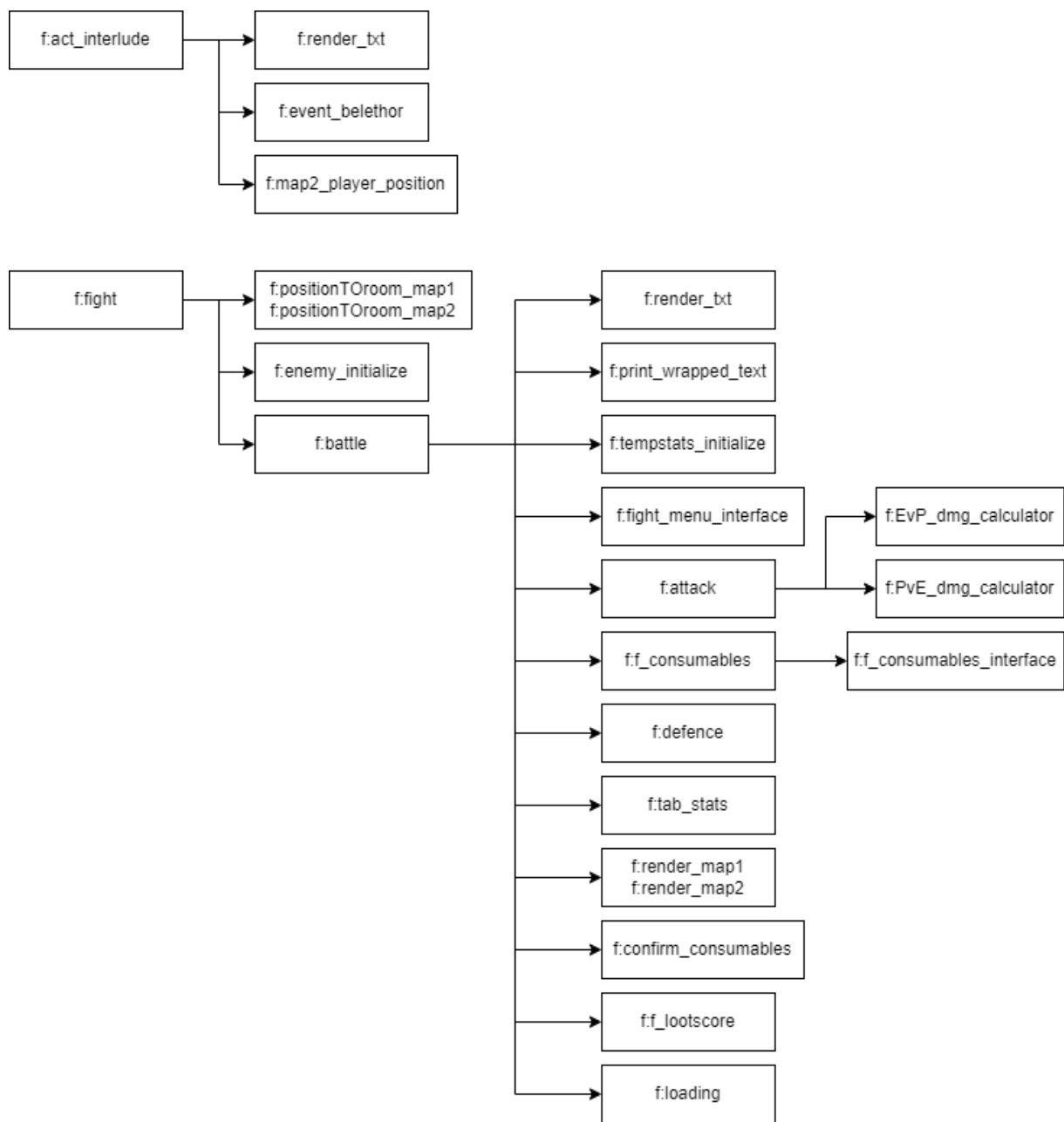
- `void tab_stats_mod(character* player);`

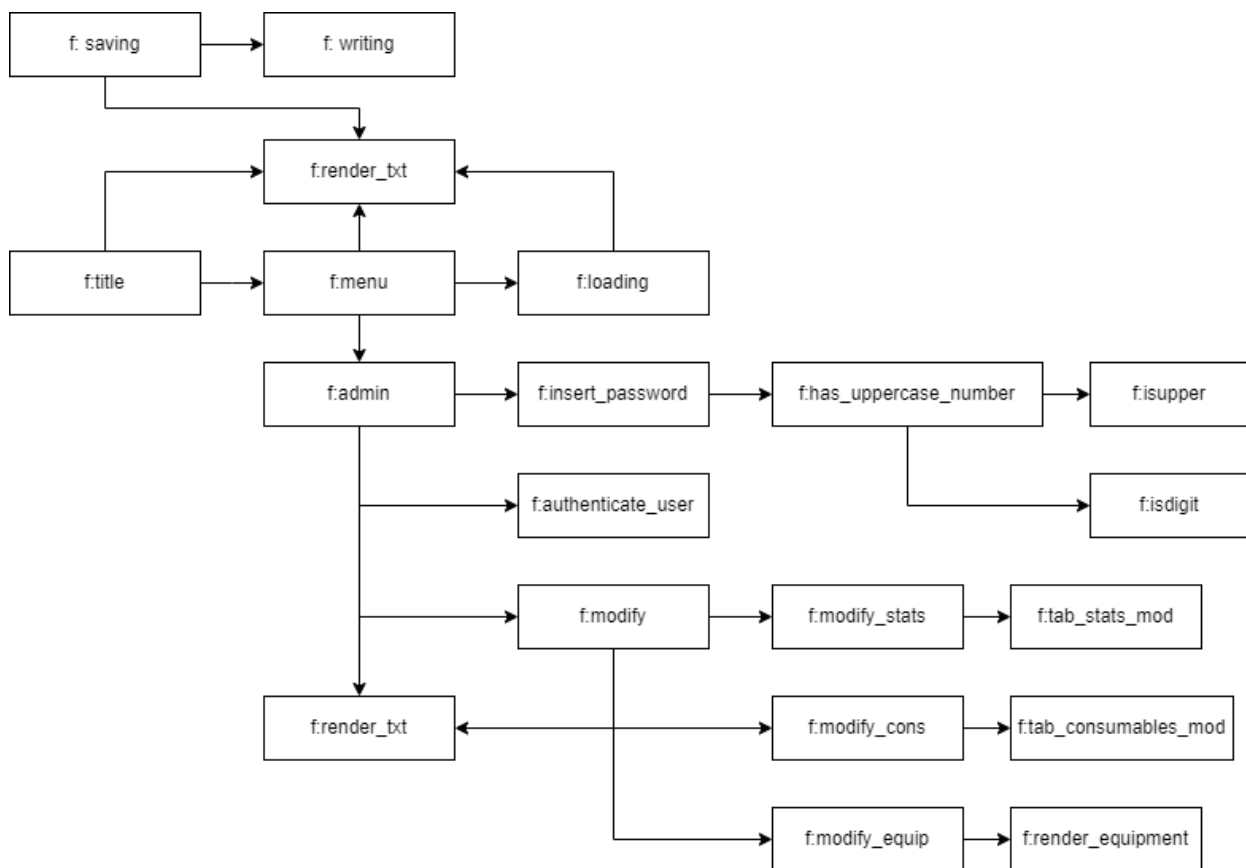
Funzione per mostrare la tabella dei consumabili

- `void tab_consumables_mod(character* player1);`

2.5 Dipendenze tra funzioni



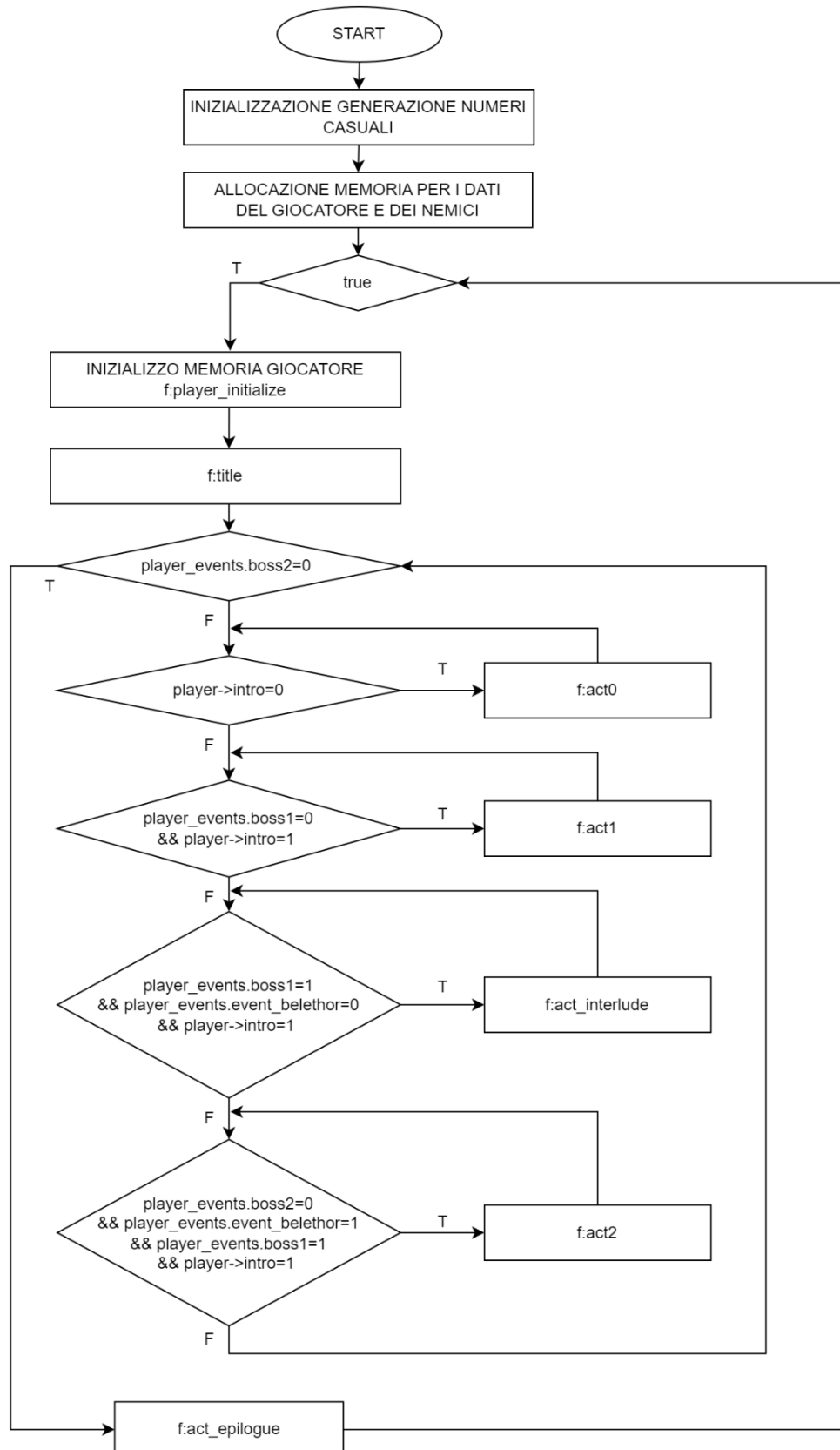


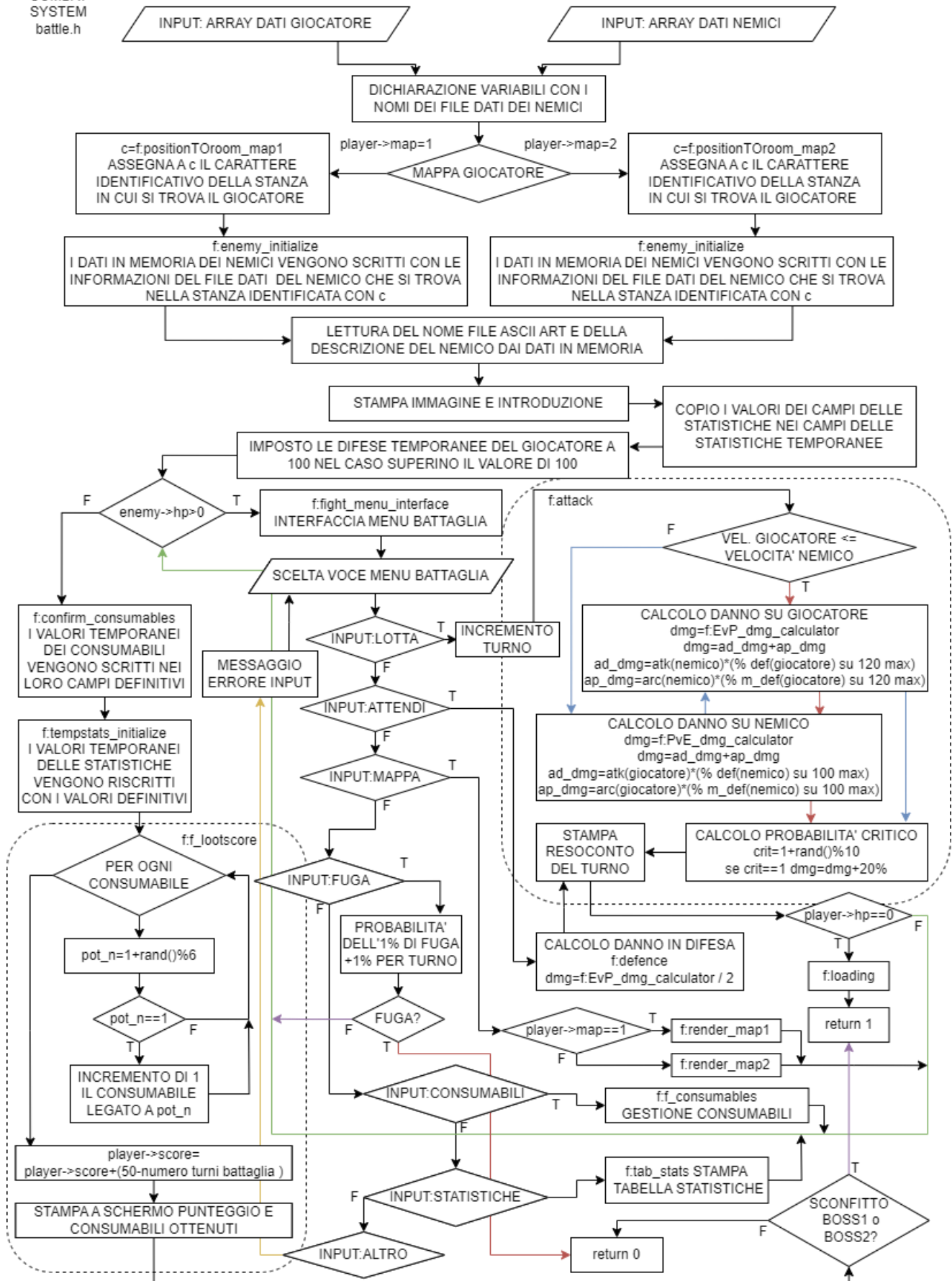


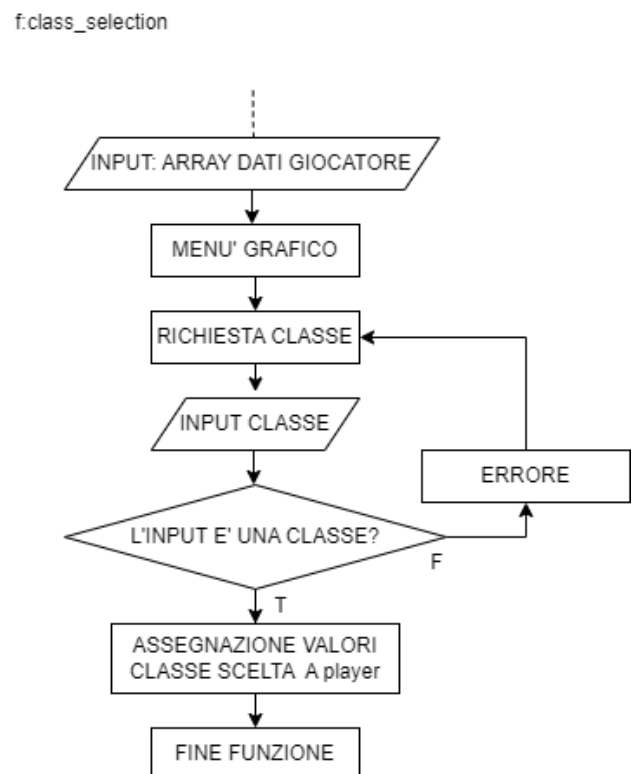
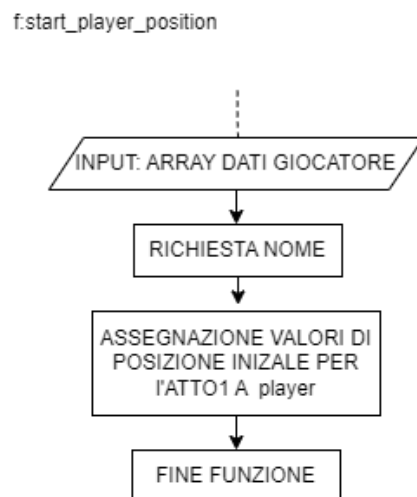
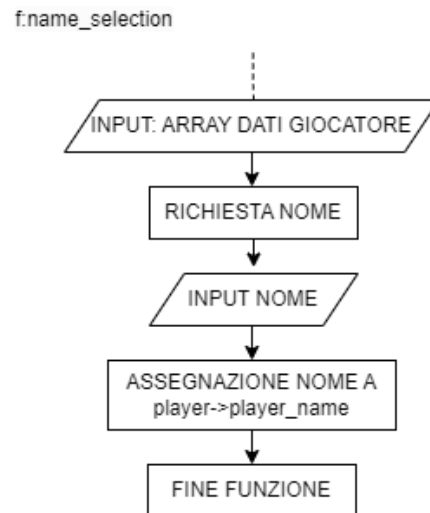
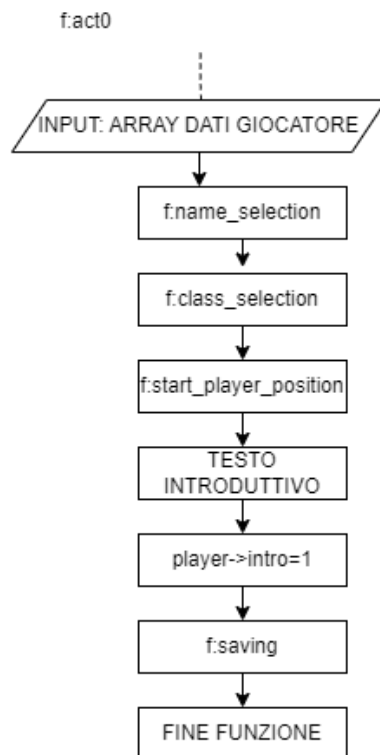
2.6 Flow Charts

All'interno di questa sezione ci saranno i flow-charts del main e delle principali funzioni. Ad esclusione del main e delle funzioni più importanti, vista la complessità di alcune di esse, l'alto numero (sono oltre 60) e la ridondanza di altre è stato necessario effettuare una selezione e una semplificazione dei flow-charts senza snaturarne il contenuto.

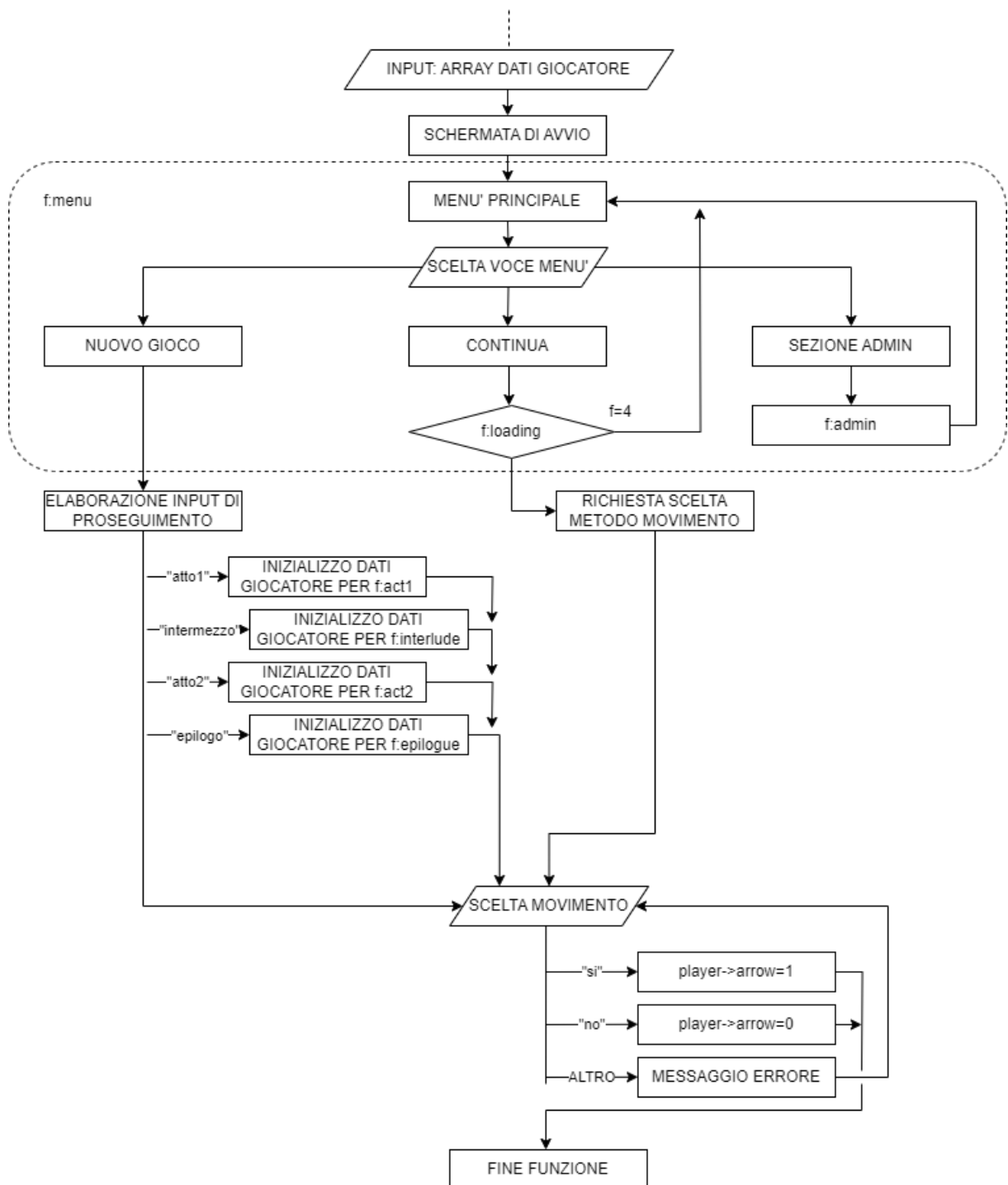
main.c



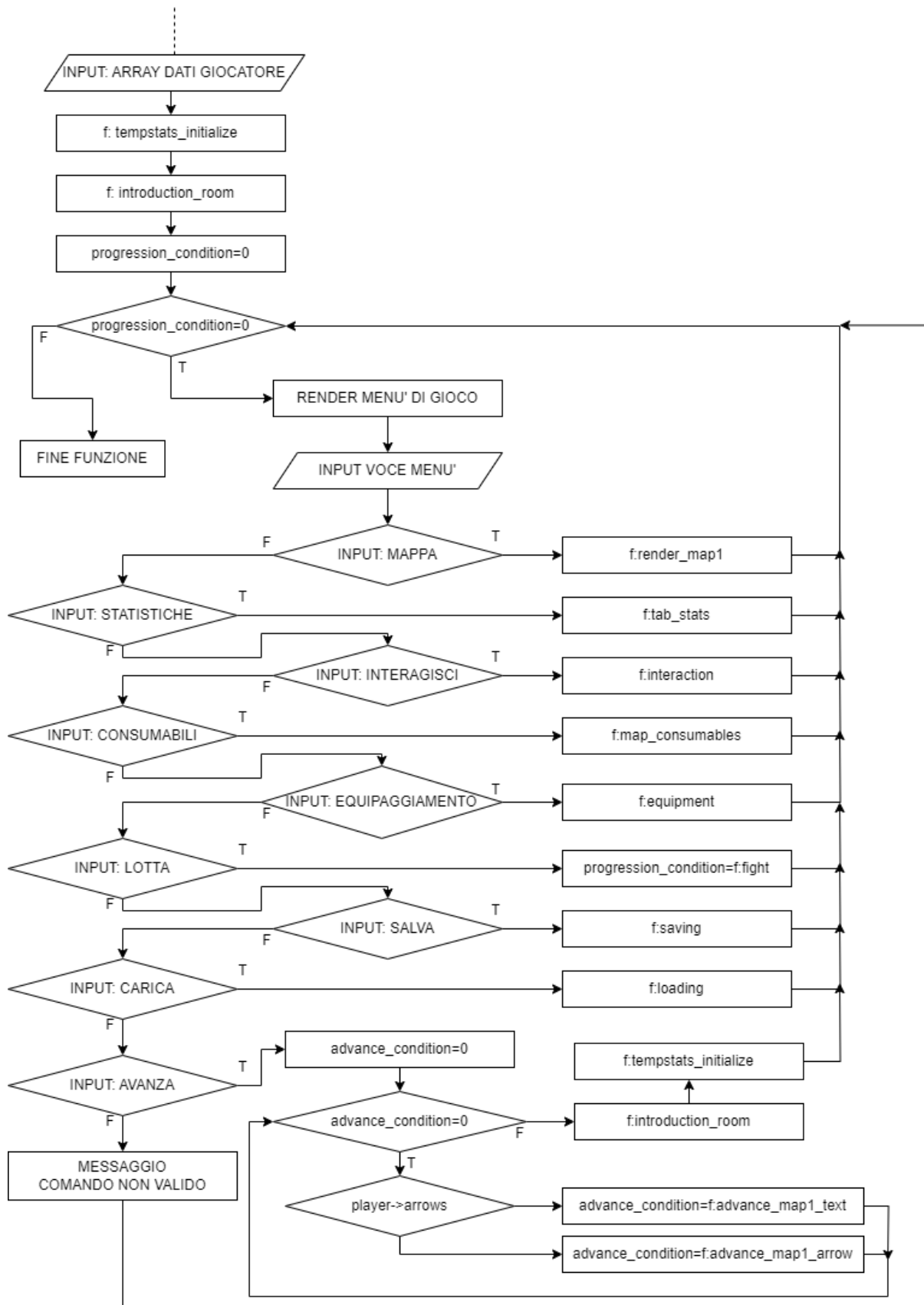




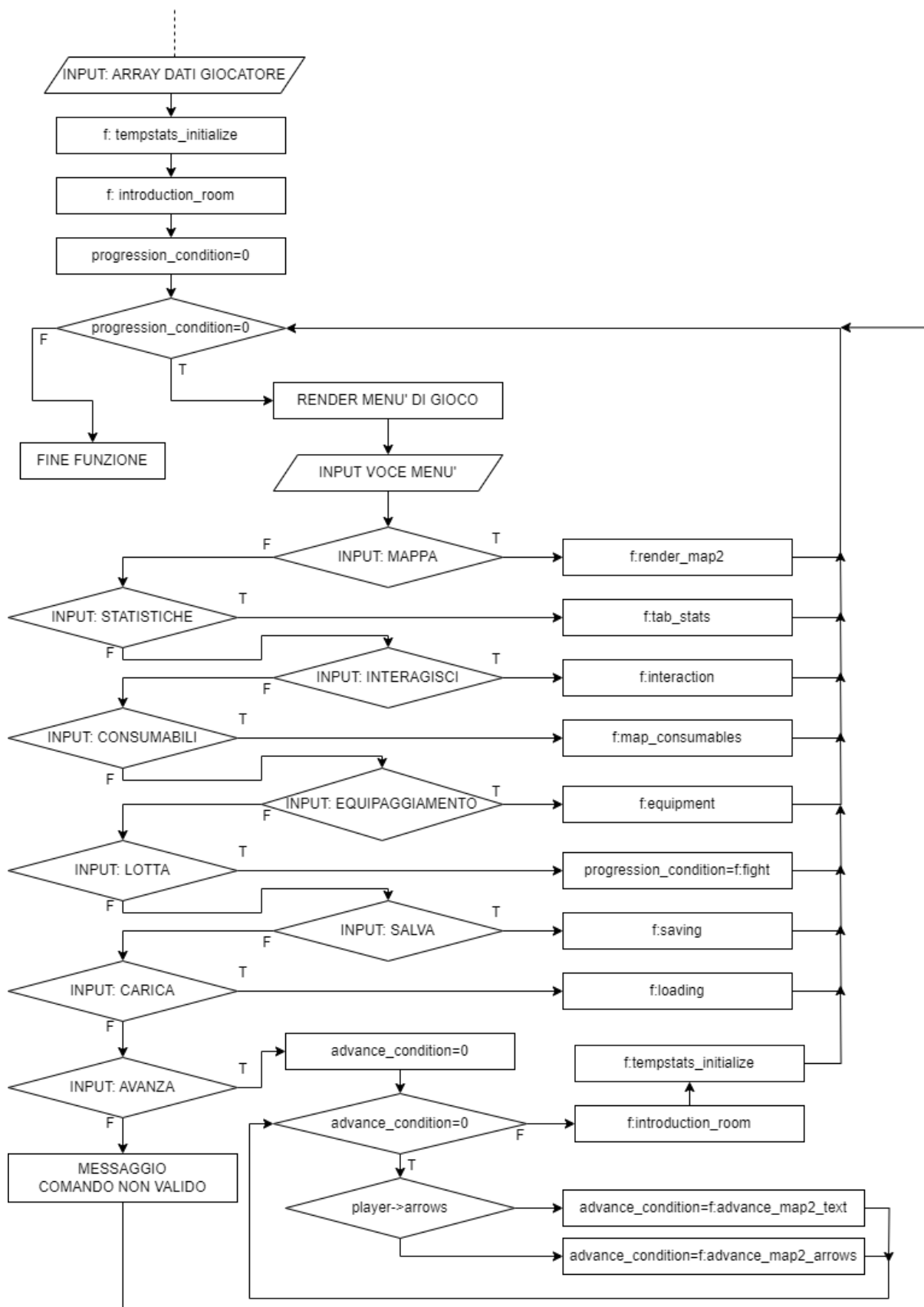
f.title

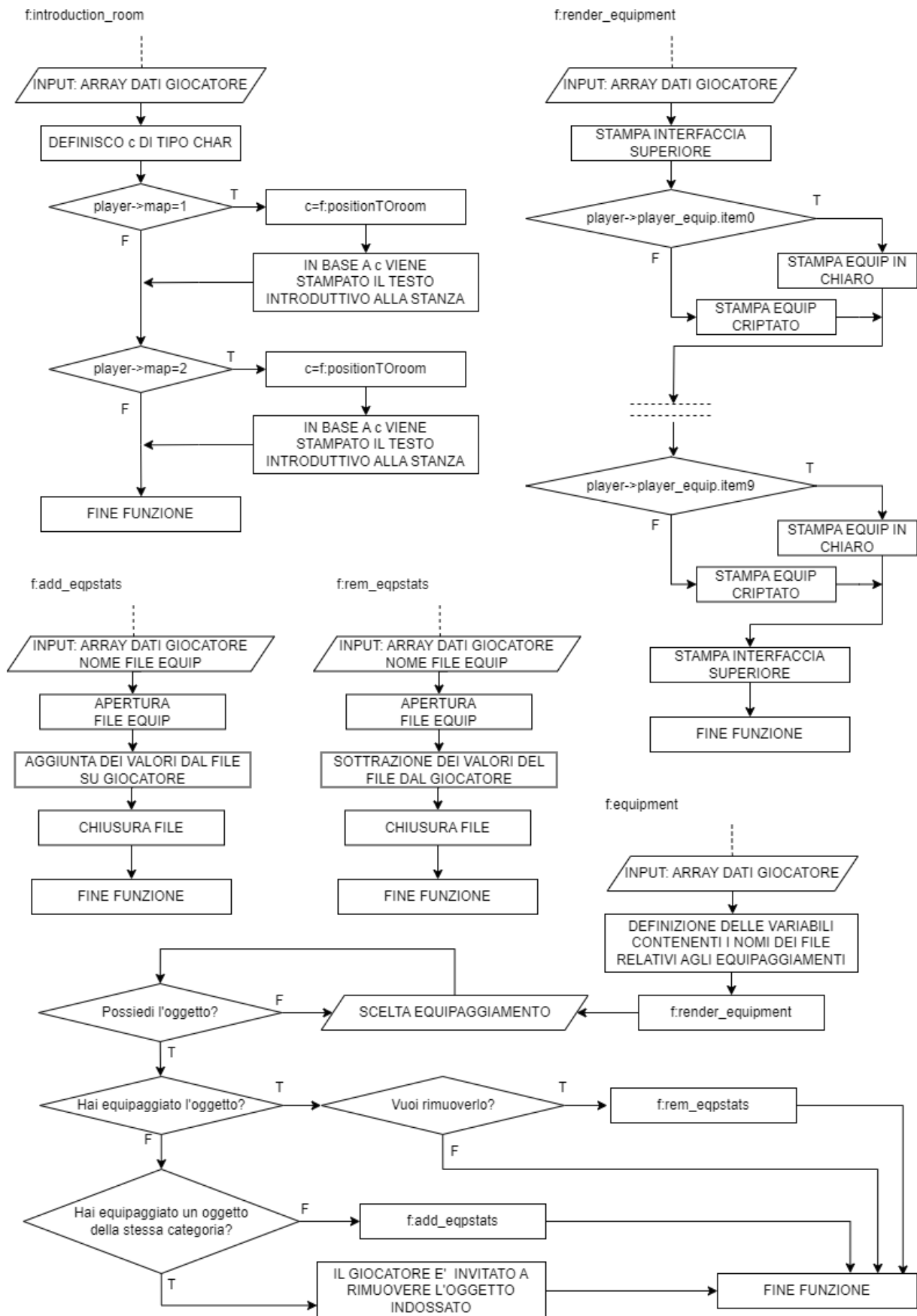


f.act1



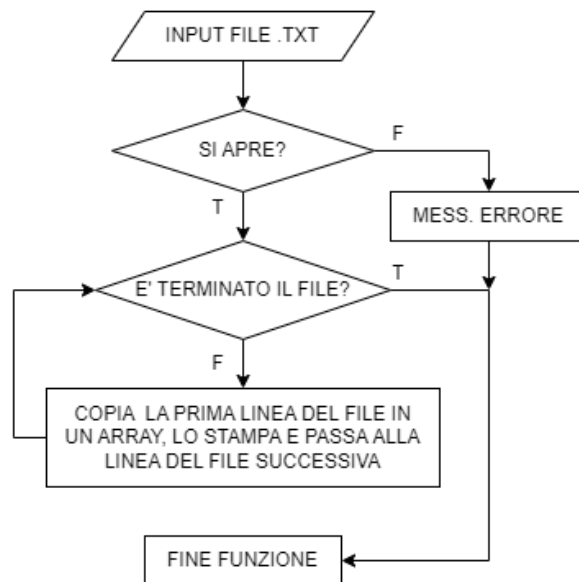
f.act2



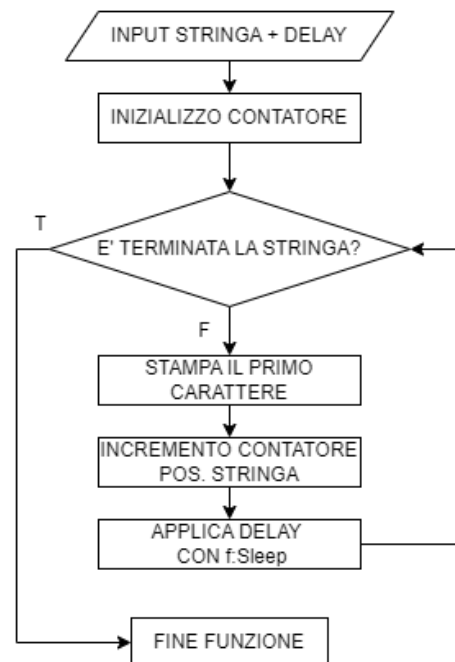


utility.h

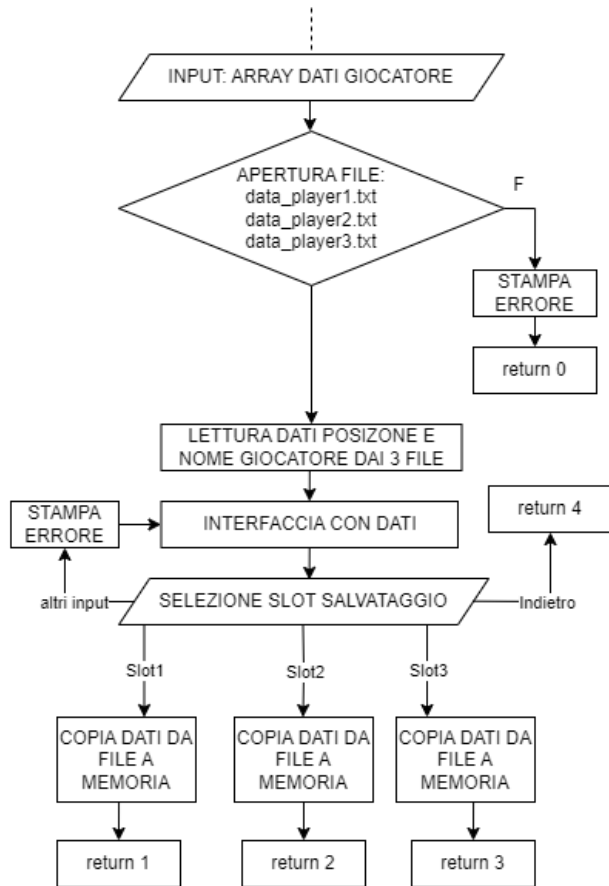
f.render_txt



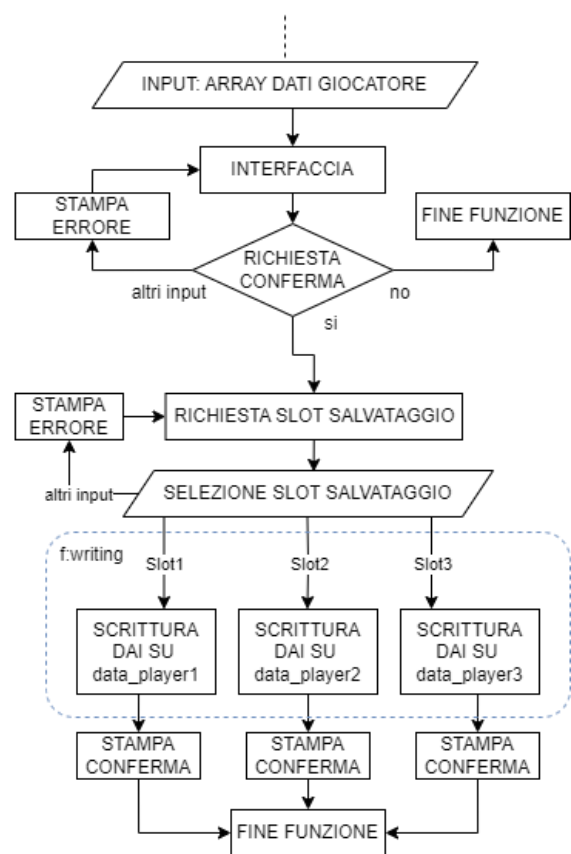
f.print_delay



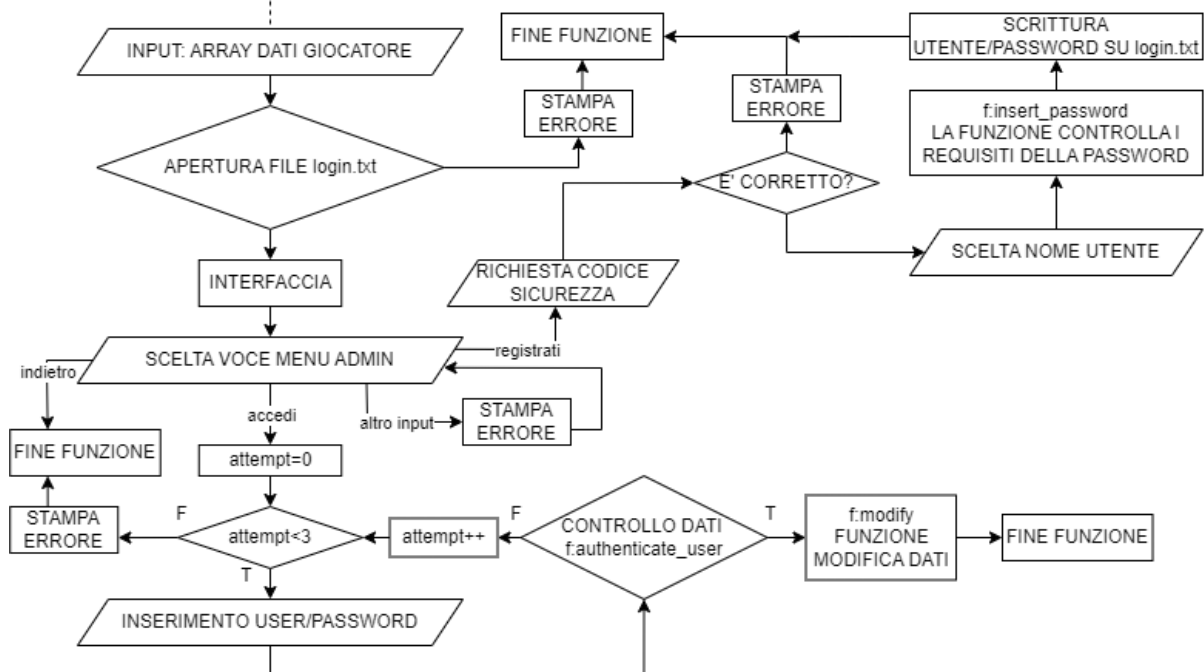
f.loading



f.saving



f.admin



3. Codifica

In questa sezione abbiamo scelto di mostrare alcune funzioni o funzionalità di particolare interesse sulla base dell'utilizzo di funzioni o soluzioni particolari per affrontare determinate sfide incontrate. Come noterete per la scrittura del codice abbiamo fatto ampio utilizzo di funzioni prese dalle librerie C++ per sopperire alle mancanze di funzioni in C.

3.1 Opzione “Avanza” del menu di gioco e le funzioni:

`advance_map1_arrows()` / `advance_map2_arrows()`

Questa funzionalità scritta parzialmente in C e parzialmente in C++ è interessante per l'uso di `GetAsyncKeyState()`, una funzione delle librerie di C++ che agisce come un interruttore booleano, alla pressione del tasto che viene dato come argomento la funzione restituisce il valore `true` permettendo di usarlo come condizione per elaborare parti specifiche di codice.

Il contesto in cui agisce `GetAsyncKeyState()` è quello dello spostamento del giocatore all'interno di una mappa, tale spostamento è gestito in parte dalla funzione `act_x()` ed in parte dalla funzione `advance_mapx_arrows()` con `x` che è il numero dell'atto.

Per evitare di far fare molteplici passaggi all'utente per effettuare movimenti consecutivi si è scelto di inserire `advance_mapx_arrows()` all'interno di un loop in `act_x()` e fruttarne il ritorno per uscire dal loop ed interrompere lo spostamento.

Durante lo sviluppo però abbiamo riscontrato alcuni problemi che siamo riusciti in parte ad aggirare.

```
while (dir_condition == 0) {
    if (GetAsyncKeyState(VK_UP)) {
        temp_y--;
        var = check_walls_map1_print(player, temp_x, temp_y);
        if (var == 1) {
            player->x = temp_x;
            player->y = temp_y;
        }
        Sleep(250);
        return 0;
    }
}
```

Lo stralcio di codice che vedete inizialmente non prevedeva la riga contenente `Sleep(250)`, come risultato avevamo uno spostamento del giocatore incontrollato fino allo scontro del giocatore con un muro che ne avesse impedito il proseguimento (vedi la funzione `check_walls_mapx_print()`). Questo perché l'elaborazione dell'`if` è enormemente più veloce del tempo impiegato dall'utente per rilasciare il tasto; quindi, il programma agiva come se l'utente desse molteplici input creando quindi il problema descritto sopra.

Per risolvere ciò abbiamo utilizzato una funzione delle librerie di C++, `Sleep()` che accetta come argomento un numero intero che indica il tempo in millisecondi durante il quale il programma deve rimanere in stallo prima di proseguire

l'elaborazione, impostando tale funzione a 250ms abbiamo notato come questo arco di tempo era sufficiente affinché l'utente potesse tranquillamente togliere la pressione del proprio dito dal tasto e quindi evitare spostamenti involontari. Un ulteriore problema inizialmente inaspettato era quello di vedere attivate voci di menu o eventi al termine dello spostamento.

Approfondendo l'uso combinato di `GetAsyncKeyState()` e `Sleep()` abbiamo scoperto come durante la fase di spostamento gli input, fossero essi previsti o no, venivano immessi in un buffer contenente già i precedenti input.

Alla prima attesa di un input da parte del programma essi venivano elaborati in maniera sequenziale.

Per risolvere ciò abbiamo provato a svuotare il buffer degli input in molteplici modi, tra cui la funzione `fflush()`, uno `scanf` che terminasse al termine dei caratteri presenti nel buffer, ecc.

Purtroppo, nessuno di questi ha funzionato. Probabilmente è un problema di buffer legato alla periferica della tastiera per la quale non abbiamo gli strumenti adatti a manipolare. L'unica soluzione che quindi abbiamo trovato è stata quella di utilizzare tasti che non interagissero con il programma da qui l'uso del tasto TAB per terminare lo spostamento. Un effetto collaterale comunque rimane, al ritorno al menu di gioco troveremo una stringa già stampata a schermo contenente i caratteri presenti nel buffer.

3.2 Funzione: print_wrapped_text()

```
void print_wrapped_text(char* text) {
    char* token = strtok(text, " ");
    int line_count = 0;
    char line[100 + 1];
    memset(line, 0, 100 + 1);
    while (token != NULL) {
        if (strlen(line) + strlen(token) + 1 > 100) {
            printf("%s\n", line);
            memset(line, 0, 100 + 1);
            line_count++;
        }
        strcat(line, token);
        strcat(line, " ");
        token = strtok(NULL, " ");
    }
    if (strlen(line) > 0) {
        printf("%s\n", line);
        line_count++;
    }
}
```

Questa funzione è l'esempio di come l'uso di funzioni delle librerie C++ ci abbia agevolato nella manipolazione di una stringa per adattarla all'interno della finestra del prompt. Essa riceve una stringa in input una stringa `text` e la stampa a schermo suddividendola in linee di massimo 100 caratteri ciascuna.

Per comprendere meglio il suo funzionamento analizziamo alcune funzioni utilizzate:

strtok(text, " "): La funzione `strtok` viene utilizzata per suddividere la stringa `text` in token, utilizzando lo spazio come delimitatore. La prima chiamata a `strtok` prende come primo parametro la stringa `text` e come secondo parametro il delimitatore " " (spazio). Questa chiamata restituisce il primo token trovato nella stringa. Le chiamate successive a `strtok(NULL, " ")` vengono utilizzate per ottenere i token successivi fino alla fine della stringa. Ad ogni chiamata successiva a `strtok`, il primo parametro viene impostato a `NULL` per indicare che la funzione deve continuare la ricerca del token successivo nella stringa originale.

memset(line, 0, 100 + 1): La funzione `memset` viene utilizzata per impostare tutti i byte dell'array `line` a 0. L'array `line` ha dimensione `100 + 1` per poter contenere fino a 100 caratteri e il terminatore di stringa `\0`.

strlen(line) + strlen(token) + 1 > 100: Questa condizione viene utilizzata per controllare se aggiungendo il token corrente alla linea attuale, la lunghezza supererebbe i 100 caratteri. Se la condizione è vera, significa che la linea è piena e deve essere stampata.

strcat(line, token): La funzione `strcat` viene utilizzata per concatenare il token corrente alla fine della stringa `line`.

if (strlen(line) > 0): Questa condizione viene utilizzata per controllare se c'è ancora una linea parziale (con almeno un carattere) da stampare alla fine dell'elaborazione dei token. Se la condizione è vera, significa che la linea parziale deve essere stampata.

3.3 Interfaccia Grafica e Audio

Non avendo nozioni su come programmare un'interfaccia grafica completa, si è cercato di simularne una utilizzando l'ASCII art che sfrutta i caratteri ASCII per creare dei disegni. Per far ciò abbiamo utilizzato una funzione creata appositamente chiamata: `render_txt()` oppure abbiamo direttamente stampato del testo appositamente formattato.

3.3.1 Funzione `render_txt()`

Questa funzione è stata creata per leggere un'immagine in ASCII art presente in un file il cui nome viene dato come argomento.

```
void render_txt(char* text) {
    string line = "";
    ifstream infile;
    infile.open(text);
    if (infile.is_open())
    {
        while (getline(infile, line))
        {
            cout << line << endl;
        }
    }
    else
    {
        cout << "fallito" << endl;
        cout << "niente" << endl;
    }
    infile.close();
    return;
}
```

Analizziamo nel dettaglio alcune parti di questo codice:

`string line = ""`: Viene dichiarata una variabile di tipo stringa chiamata `line` e inizializzata come una stringa vuota. Questa variabile verrà utilizzata per memorizzare ciascuna riga del file di testo.

`ifstream infile`: Viene dichiarato un oggetto di tipo `ifstream`, che rappresenta un file di input. Questo oggetto verrà utilizzato per aprire e leggere il file di testo.

`infile.open(text)`: La funzione `open` dell'oggetto `infile` viene utilizzata per aprire il file di testo specificato dal parametro `text`. `text` è il nome del file di testo passato alla funzione

`if (infile.is_open())`: Questa condizione viene utilizzata per verificare se il file di testo è stato aperto correttamente.

`while (getline(infile, line))`: La funzione `getline` viene utilizzata all'interno di un ciclo `while` per leggere ciascuna riga del file di testo. La funzione `getline` legge una linea intera dal file di testo rappresentato dall'oggetto `infile` e la memorizza nella variabile `line`. Il ciclo `while` continua finché ci sono ancora righe nel file.

`cout << "fallito" << endl;`: Se il file di testo non è stato aperto correttamente, viene stampata la stringa "fallito" a schermo.

3.3.2 Funzione render_map1()

Essendo la mappa un'immagine che cambia in funzione della posizione del giocatore, ho inizialmente ricreato la mappa con l'uso dei printf() in seguito ho ricostruito ogni riga della mappa con le dovute modifiche in base alla posizione attuale del giocatore, in questo modo si riesce ad ottenere una mappa sempre coerente.

```
void render_map1(character*player) {
    int x = player->x;
    int y = player->y;
    printf("\033[1;31;47m MAPPA DELLE CATACOMBE DI MIST \033[0m\n");
    printf("          ,---,-----,\n");
    if (x == 15 && y == 2) {
        printf("          | x |          |\n");
    }
    else if (x == 16 && y == 2) {
        printf("          |   | x       |\n");
    }
    else if (x == 17 && y == 2) {
        printf("          |   |      x   |\n");
    }
    else if (x == 18 && y == 2) {
        printf("          |   |      x  |\n");
    }
}
```

3.3.2 Funzione PlaySound()

La funzione PlaySound() è una funzione delle API di Windows che consente di riprodurre file audio utilizzando diverse opzioni di riproduzione.

```
PlaySound(TEXT("catacombs.wav"), NULL, SND_LOOP | SND_ASYNC);
```

Essa riceve come argomento il nome del file audio .wav, il secondo argomento rappresenta l'handle dell'istanza dell'applicazione che sta chiamando la funzione PlaySound() che viene in genere impostato su NULL per indicare l'istanza corrente dell'applicazione, gli ulteriori argomenti riguardano altre opzioni di riproduzione. SND_SYNC: la funzione PlaySound() attende la fine della riproduzione prima di restituire il controllo. SND_ASYNC: la funzione PlaySound() restituisce immediatamente senza attendere la fine della riproduzione. SND_LOOP: Il file audio viene riprodotto in loop.

3.4 Funzione: insert_password();

```
void insert_password(char *v) {  
  
    char c;  
    int i = 0, length = 0, check=0;  
    do {  
  
        printf("\nPassword:\nMinimo %d caratteri, massimo %d caratteri;\nAlmeno  
una lettera maiuscola;\nAlmeno un numero.\n : ", min_password_lenght,  
max_password_lenght);  
  
        while ((c = _getch()) != 13) { //Il numero 13 è l'ASCII dell'invio  
            v[i] = c;                // *Questo while permette  
            i++;                    // *l'oscuramento della password  
            printf("*");            // *durante la digitazione dell'utente  
        }  
  
        v[i] = '\0';  
        length = strlen(v); //funzione della libreria string.h che permettedi  
vedere la lunghezza di una stringa  
        check = has_uppercase_number(v); //funzione che permette di controllare  
se ci sono maiuscole e numeri  
  
        if (length < 8 || length > 20 || !check) { //controllo delle quattro  
condizioni della password  
            printf("\nLa stringa non rispetta le dimensioni o le caratteristiche  
richieste. Riprova.\n");  
        }  
  
    } while (length < 8 || length > 20 || !check); //controllo delle quattro  
condizioni della password  
}
```

Questa funzione è molto utile per poter mettere qualsiasi condizione su una stringa che si vuole controllare. In questo caso ho messo quattro condizioni, ovvero:

1. La stringa deve avere più di 8 caratteri;
2. La stringa deve contenere un massimo di 20 caratteri;
3. La stringa deve contenere almeno una lettera maiuscola;
4. La stringa deve contenere almeno un numero.

Molto utile nella funzione è stato il seguente ciclo while:

```
while ((c = _getch()) != 13) { //Il numero 13 è l'ASCII dell'invio  
    v[i] = c;                // *Questo while permette  
    i++;                    // *l'oscuramento della password  
    printf("*");            // *durante la digitazione dell'utente  
}
```

Questo ciclo, infatti, permette di oscurare l'input della password quando viene inserita dall'utente. In questo modo abbiamo applicato anche una metodologia di privacy all'interno del programma. Un altro pezzo molto fondamentale è la funzione `has_uppercase_number()`, che verrà spiegata successivamente.

3.5 Funzione: has_uppercase_number();

```
int has_uppercase_number(char *str) {
    int uppercase = 0;
    int number = 0;

    while (*str != '\0') { //fino a quando la stringa non finisce
        if (isupper(*str)) //controllo delle eventuali lettere maiuscole(funzione
//proveniente da ctype.h)
            uppercase = 1;
        if (isdigit(*str)) //controllo degli eventuali numeri maiuscole(funzione
//proveniente da ctype.h)
            number = 1;

        if (uppercase && number) //se sono presenti almeno una maiuscola e almeno
//un numero allora ritorni il vero
            return 1;

        str++; //aumento del valore dell'indirizzo della stringa
    }

    return 0;
}
```

Questa funzione, seppur molto semplice, contiene due funzioni molto importanti utili per la funzione principale. Entrambe provengono dalla libreria `ctype.h` e sono `isupper()` e `isdigit()`. Entrambe queste funzioni prendono in ingresso un intero (in questo caso la stringa inserita dall'utente parte dall'inizio e avanza un carattere alla volta `str++`) corrispondente al codice ASCII di un carattere e restituisce un valore diverso da zero se il carattere è una lettera maiuscola (nel caso di `isupper()`) o è un numero (nel caso di `isdigit()`), altrimenti restituisce zero.

5. Test

4.1 Come è stata condotta la fase di Testing

Vista la complessità del codice e le molteplici dipendenze, la fase di testing è stata affrontata appena il codice era tale da permettere di provare parte del gioco e vedere fin da subito problemi che, se scoperti in fase avanzata potevano rendere difficile l'origine degli stessi.

Il gioco quindi è stato fatto provare da più persone in più fasi dello sviluppo.

Le molteplici dipendenze rendono impossibile testare singolarmente ogni funzionalità in modo indipendente, oltretutto questo approccio avrebbe nascosto problematiche legate al passaggio dei dati tra le varie funzioni che le sfruttano per scopi diversi. Questi test sono quindi frutto di sessioni di gioco dalle quali sono stati estrapolati i dati ed inseriti in tabella, ricordando che essi sono inerenti alla versione definitiva del gioco e non esaurienti dell'intera fase di testing che il gioco ha subito.

Codice Requisito	Codice Test	Nome	Descrizione Test	Eventuale Input	Risultato Atteso	Risultato Ottenuto
RB01 RB02 RB03 RB13 RB14	da B1.1 a B1.30+	Caricamento Nemico Inizializzazione Valori Battaglia Interfaccia Menu Battaglia Battaglia Caricamento Battaglia	Il test consiste nell'iniziare una battaglia in ogni stanza contenente un nemico.		Il gioco stampa a schermo le informazioni del nemico previsto, avvia la musica e stampa a schermo il menu battaglia	Il gioco stampa a schermo le informazioni del nemico previsto, avvia la musica e stampa a schermo il menu battaglia
RB01 RB02 RB03 RB13 RB14	da B2.1 a B2.8	Caricamento Nemico Inizializzazione Valori Battaglia Interfaccia Menu Battaglia Battaglia Caricamento Battaglia	Il test consiste nel testare la voce "Lotta" del menù di gioco in luoghi in cui non è prevista una battaglia.		Il gioco dovrebbe avvisare l'assenza di nemici nell'area	Il gioco avvisa dell'assenza di nemici nell'area
RB13	da B3.1 a B3.12	Battaglia	Elaborazione delle scelte del giocatore	Testati tutti Input Validi: "lotta", "Lotta", "mappa", "Mappa", "fuga", ecc.	Il gioco elabora la scelta corretta.	Il gioco elabora la scelta corretta.
RB13	B5-6	Battaglia	Elaborazione delle scelte del giocatore	Testati Input non Validi "snjad",	Il gioco avvisa di aver immesso un comando non valido	Il gioco avvisa di aver immesso un comando non valido

				"niooj34", ecc.	e ripropone il menù Battaglia	e ripropone il menù Battaglia
RB04 RB06 RB09 RB10 RB11 RB12 RB13	B7	Tabella delle Statistiche Menu Consumabili in Battaglia Calcolo Danni PvE Calcolo Danni EvP Calcolo Danni in Difesa Turno di Gioco Battaglia	Il test è stato condotto utilizzando le varie classi, in diverse partite e testandole in almeno una battaglia per ogni tipo di nemico. Sono stati utilizzati Consumabili per controllare che le modifiche alle statistiche agissero sul calcolo dei danni, così come sulla priorità del turno.		Il gioco è programmato per calcolare i danni in base ad una percentuale di penetrazione, con la probabilità del 10% di colpo critico, con il dimezzamento del danno ricevuto in attesa, con il primo colpo inflitto dal personaggio più agile.	Il gioco ha sempre elaborato la battaglia secondo i parametri previsti.
RB04 RB05 RB06 RB07 RB08 RG07	BG1	Tabella delle Statistiche Interfaccia Consumabili Menu Consumabili in Battaglia Aggiornamento Consumabili Sistema di Loot Punteggio post Battaglia Menu Consumabili in Esplorazione	Il test è stato condotto per controllare che le quantità dei consumabili tra una battaglia e l'esplorazione fossero coerenti, che portando al massimo una statistica ne venisse limitato l'uso e che il drop dei consumabili rientrasse nei parametri cioè del 16,6% per ogni tipologia e che il punteggio venisse regolarmente aggiornato.		Il gioco elabora i consumabili secondo i parametri dello sviluppatore.	Il gioco elabora i consumabili secondo i parametri dello sviluppatore.
RG13 RG11 RG10 RG12 RG13 RG14 RB04	G1	Atto 0 Selezione Classe Giocatore Scelta Nome Giocatore Imposta Posizione Iniziale Atto 1 Stampa Mappa Catacombe Tabella delle Statistiche	Il test è stato condotto per controllare la gestione dell'Atto Introduttivo e che i dati fossero elaborati per iniziare l'Atto 1. Per le parti in cui ci sono scelte da intraprendere o dati da inserire, sono stati usati input e stringhe secondo i parametri previsti. Per controllare queste informazioni è stato fatto uso della Mappa e delle Statistiche visualizzabili nell'Atto 1.		Il gioco permette di giocare l'intero atto introduttivo, nei dialoghi viene correttamente stampato il suo nome, assegna ad esso i valori relativi alla classe, ed una volta iniziato l'atto 1 il programma posiziona il giocatore secondo il parametro previsto.	Il gioco permette di giocare l'intero atto introduttivo, nei dialoghi viene correttamente stampato il suo nome, assegna ad esso i valori relativi alla classe, ed una volta iniziato l'atto 1 il programma posiziona il giocatore secondo il parametro previsto.
RG10	G2	Scelta Nome Giocatore	Test nome lungo 50+ Caratteri	Stringa da oltre 50 Caratteri	Situazione non prevista. Possibile Crash del gioco.	Il gioco elabora un input non corretto e chiede conferma di un nome errato.
RG11	G3	Selezione Classe Giocatore	Scelta di una classe non prevista o	"mago" "dasdh13" "monaco"	Il programma avvisa di non aver riconosciuto la classe scelte e richiede	Il programma avvisa di non aver riconosciuto la classe scelte e richiede

			stringa non corretta.		all'utente di inserire nuovamente la classe.	all'utente di inserire nuovamente la classe.
RG19 RG28 RG17 RG15 RG24 RG26	G4	Atto 1 Atto 2 Avanzamento con Frecce Mappa Catacombe Controllo Mura Mappa Catacombe Controllo Mura Mappa Castello Avanzamento con Frecce Mappa Castello	Il test è stato condotto per provare il sistema di movimento all'interno delle mappe di gioco. Per i dettagli sulla voce "Avanzamento" del Menu di Gioco leggere la sezione 3.1 del capitolo sulla Codifica.		Il programma permette il movimento solo nelle aree accessibili, la mappa aggiorna in tempo reale la posizione del giocatore all'interno di essa e nel caso si provi ad uscire dai confini previsti viene visualizzato un messaggio su tale azione.	Il programma permette il movimento solo nelle aree accessibili, la mappa aggiorna in tempo reale la posizione del giocatore all'interno di essa e nel caso si provi ad uscire dai confini previsti viene visualizzato un messaggio su tale azione.
RI01-10 RBo5 RG03 RBo4 RG08	I1.1	Elaborazione Casse Interfaccia Consumabili Interfaccia Equipaggiamento Tabella delle Statistiche Gestione delle Interazioni	Il test è stato condotto per provare il funzionamento del sistema delle Casse. Questo prevede l'avvio corretto di tali eventi nelle stanze ove presente una cassa al suo interno e il successivo controllo dell'inventario (consumabili + equipaggiamento) e del punteggio, come conferma dell'avvenuta apertura.		Il programma stampa a schermo l'immagine di una cassa e relativa musica, viene avvisato il giocatore del contenuto che viene aggiunto al suo inventario.	Il programma stampa a schermo l'immagine di una cassa e relativa musica, viene avvisato il giocatore del contenuto che viene aggiunto al suo inventario.
RI01-10	I1.2	Elaborazione Casse	Controllo delle stanze con le casse già aperte.	Voce "Interagisci" nelle stanze previste dal test.	Il gioco avvisa che la cassa contenuta nella stanza è stata già aperta.	Il gioco avvisa che la cassa contenuta nella stanza è stata già aperta.
RG06 RG03 RG04 RG05 RBo4 RG19 RG28	G5	Menu Equipaggiamento Interfaccia Equipaggiamento Aggiunta Equipaggiamento Rimozione Equipaggiamento Tabella delle Statistiche Atto1 Atto2	Il test è stato condotto per provare il sistema di equipaggiamento. Durante il test è stato provato in maniera randomica l'aggiunta di equipaggiamento, la sua rimozione, la limitazione di aggiunta di più oggetti equipaggiabili dello stesso tipo e il controllo tramite la tabella delle statistiche delle azioni.		Il gioco deve gestire in maniera corretta tutte le funzioni, mostrando correttamente l'interfaccia grafica, avvisando in caso di aggiunta di più oggetti e aggiornando le statistiche.	Il gioco deve gestire in maniera corretta tutte le funzioni, mostrando correttamente l'interfaccia grafica, avvisando in caso di aggiunta di più oggetti e aggiornando le statistiche.
Vedi voci del test G.1.5	G6	Vedi voci del test G.1.5	Il test è stato condotto per testare le eccezioni nel sistema di equipaggiamento. Per far ciò come input sono stati scelti pezzi di equipaggiamento non		Il gioco riconosce se la stringa inserita non è valida o si tratta di un oggetto non ancora ottenuto e	Il gioco riconosce se la stringa inserita non è valida o si tratta di un oggetto non ancora ottenuto e

			ancora trovati e stringhe non valide.		stampa il relativo messaggio di avviso.	stampa il relativo messaggio di avviso.
RB13	B8	Battaglia	Test sul funzionamento della fuga da una battaglia, per fare ciò è stata selezionata più volte la voce "fuga" del menu battaglia per controllare che la fuga avvenga con una probabilità di 1 su 100 e con l'aumento di tale probabilità con l'aumento dei turni di una battaglia.		Il gioco ha permesso la fuga secondo i parametri previsti.	Il gioco ha permesso la fuga secondo i parametri previsti.
RG21 RG22	G7	Evento Intermezzo Atto Intermezzo	Questo test è simile al test G1.8 poiché nell'intero atto vi è solo un'interazione dovuta all'evento intermezzo. Il test è stato condotto per vedere come l'evento si comporta a diverse combinazioni di input.		Il gioco solo con la giusta combinazione permette la prosecuzione, altrimenti viene data la possibilità di ripetere l'evento.	In tutti i test il programma si è sempre attenuto ai parametri dettati in fase di sviluppo.
RG29	G9	Atto Epilogo	L'Epilogo non ha interazioni con l'utente quindi il test è stato condotto per vedere la buona resa grafica a schermo ed eventuali problemi di crash inattesi.		Il gioco mostra l'epilogo senza alcun problema e secondo i parametri dettati in fase di sviluppo.	Il gioco mostra l'epilogo senza alcun problema e secondo i parametri dettati in fase di sviluppo.
RI11-20	I1	Elaborazione Eventi	Gli eventi sono principalmente di due tipi, interattivi e non interattivi. Quelli interattivi consistono nella risoluzione di un enigma che avviene tramite una stringa da dare in input. Durante le varie partite di testing abbiamo provato tutti i vari eventi.		Il programma stampa a schermo le informazioni relative agli eventi e il programma riesce a riconoscere se la stringa è la soluzione dell'evento o meno.	Il programma stampa a schermo le informazioni relative agli eventi e il programma riesce a riconoscere se la stringa è la soluzione dell'evento o meno.
RB13	B10	Battaglia	Test condotto per testare l'avvio del caricamento in seguito di un game over da parte del giocatore.		Il programma permette l'accesso alla sola voce di Caricamento e nel caso si provi l'accesso ad altre voci del menu, il giocatore viene avvisato dello stato di Game Over e viene reindirizzato al caricamento.	Il programma permette l'accesso alla sola voce di Caricamento e nel caso si provi l'accesso ad altre voci del menu, il giocatore viene avvisato dello stato di Game Over e viene reindirizzato al caricamento.
RA02 RA03	A1.1	Registrazione	Inserimento Utente e Password fuori parametri	Utente: stringa casuale di oltre 20 caratteri Password:123cba123	Viene chiesto all'utente di ritentare l'inserimento di utente e password	Viene chiesto all'utente di ritentare l'inserimento di utente e password
RA02 RA03	A1.2	Registrazione	Inserimento Utente fuori parametro e Password nei parametri	Utente: stringa casuale di oltre 20 caratteri Password:123ABC@bca	Viene accettata la registrazione ma il nome utente avrà solo i primi 20 caratteri inseriti	Viene accettata la registrazione ma il nome utente avrà solo i primi 20 caratteri inseriti

RA05	A2.1	Salvataggio	Assenza dei file di Salvataggio	-	Viene creato il file e salvati i dati.	Viene creato il file e salvati i dati.
RA05	A2.2	Salvataggio	Memoria locale piena o memoria locale corrotta	-	Viene indicato a schermo un errore sull'apertura del file.	Viene indicato a schermo un errore sull'apertura del file.
RA06	A3	Caricamento	Assenza dei file di Salvataggio	-	Viene indicato a schermo un errore sull'apertura del file.	Viene indicato a schermo un errore sull'apertura del file.
RA09 RA12	A4.1	Modifica Statistiche	Modifica Difesa oltre il limite massimo	113	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo valore	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo valore
RA09 RA12	A4.2	Modifica Statistiche	Modifica Salute	3500	La tabella con le statistiche viene aggiornata.	La tabella con le statistiche viene aggiornata.
RA10 RA13	A5.1	Modifica dei Consumabili	Modifica numero Ampolle	140	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo valore	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo valore
RA10 RA13	A5.2	Modifica dei Consumabili	Modifica numero Ampolle	80	La tabella con le quantità dei consumabili viene aggiornata.	La tabella con le quantità dei consumabili viene aggiornata.
RA11 RG03	AG1.1	Modifica Equipaggiamento	Equip. già indossato	-	Il valore in memoria non viene cambiato. L'oggetto è visibile in chiaro nell'interfaccia	Il valore in memoria non viene cambiato. L'oggetto è visibile in chiaro nell'interfaccia
RA11 RG03	AG1.2	Modifica Equipaggiamento	Equip. non ottenuto	-	Viene posto a 1 il valore del campo corrispondente in memoria e l'oggetto diventa visibile nell'interfaccia	Viene posto a 1 il valore del campo corrispondente in memoria e l'oggetto diventa visibile nell'interfaccia
RA11 RG03	AG1.3	Modifica Equipaggiamento	Equip. ottenuto	-	Non viene effettuata alcuna modifica	Non viene effettuata alcuna modifica
RA11 RG03	AG1.4	Modifica Equipaggiamento	Equip. non esistente	Canotta Lunare	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo equipaggiamento	Viene stampato a schermo l'errore e viene richiesto di inserire un nuovo equipaggiamento
RA04	A6.1	Autenticazione utente	Dati esistenti in login.txt	Utente:asd Pass:asd	L'utente viene avvisato delle credenziali errate inserite	L'utente viene avvisato delle credenziali errate inserite
RA04	A6.2	Autenticazione utente	Dati assenti in login.txt	Utente: Test Pass:123ABC@abc	Viene dato l'accesso all'utente	Viene dato l'accesso all'utente
RA04	A6.3	Autenticazione utente	Assenza file login.txt	-	Errore a schermo sull'apertura del file	Errore a schermo sull'apertura del file

5. Conclusioni

5.1 Conclusioni

Il programma è abbastanza solido e poco incline al crash o a comportamenti inaspettati.

Sono state prese molte precauzioni per evitare conflitti e problemi di memoria.

La sua alta modularità gli permette di essere predisposto a futuri aggiornamenti senza molti sforzi e modifiche strutturali.

Il software è stato fatto testare da molte persone che hanno valutato positivamente la loro esperienza ed hanno contribuito con i loro feedback a renderlo migliore.

5.2 Possibili Future Implementazioni

- + Migliorare il Bilanciamento tramite la modifica di statistiche, quantità di oggetti distribuiti e modi in cui essi vengono distribuiti.
- + Parzialmente Implementato: Aggiungere un sistema per permettere di salvare durante una battaglia utilizzando i dati precedenti all'inizio della stessa. Il sistema d'uso dei consumabili e l'uso di statistiche temporanee nello struct del giocatore sono già predisposti per tale funzionalità.
- + Permettere di lasciare la stanza durante eventi di tipo "enigma" quando essi non prevedano un impedimento fisico del giocatore.
- + Presentazione degli eventi di gioco, tramite storyboard.
- + Limitazione della quantità di oggetti utilizzabili per turno e stabilire per ogni potenziamento la durata in turni degli stessi.
- + Sistema di selezione degli oggetti consumabili ed equipaggiamento che possa permettere di vedere in dettaglio le statistiche, le informazioni sul loro uso e la lore.
- + Ottimizzazione della dimensione del gioco, utilizzando sistemi alternativi per l'audio.
- + Creazione di un file di installazione per evitare che l'utente possa erroneamente modificare i file necessari al gioco.
- + Creare eccezioni per le stringhe di lunghezza elevata
- + Sistema avanzato di Salvataggio
- + Aggiungere eccezioni nel menù opzioni per gli input con caratteri nelle richieste numeriche
- + Aggiungere più opzioni per la modifica del proprio personaggio

5.3 Bug Noti

- + L'uso del sistema tramite frecce crea un conflitto con il buffer della tastiera. I tasti in precedenza premuti potrebbero interferire con il movimento. In ugual maniera, la successiva richiesta di un comando testuale viene influenzata dai tasti premuti per muoversi che mostra a schermo il vecchio comando inserito modificato dagli input usati durante il movimento.

6. Guida Utente

6.1 Soluzioni agli eventi di gioco

Enigma Anagramma: il buio ci guida.

Indovinello: la luna.

Cassaforte: 5619543.

Selene: Empatizza.

Belethor: 2, 1, 1, 3.

Selene (dialogo nascosto): aver visitato la tomba di Seluvis prima di incontrare Selene

6.2 Altre Informazioni

Dati di Accesso Admin di Riserva:

Utente: Test

Password: 123ABC@abc

Accesso diretto ad un qualunque Atto della storia:

Per accedere direttamente ad un atto in particolare basterà scrivere le seguenti stringhe nel menu titolo (con l'audio del lupo) e procedere nel menu principale e iniziare una nuova partita.

“atto1” per l'Atto I

“intermezzo” per l'Atto Intermezzo

“atto2” per l'Atto II

“epilogo” per l'Atto Epilogo